

Video Intelligence: Identity & Asset Recognition

A phase-by-phase build, test and measurement plan
extending the PS-TR-1 edge-to-cloud prototype

AUTHOR	Prateek Srivastava
REPOSITORY	github.com/prateekhbtu/video-intel
SCOPE	PS-3 Person Recognition & Identity Intelligence PS-4 Object & Asset Recognition
BASELINE	PS-TR-1 prototype, post-correction
PHASES	15, approximately 38 engineering hours
STATUS	Single source of truth for development
DATE	16 August 2026

Contents

Part 0. Ground Rules

- 0.1 What this document is
- 0.2 Prerequisite state
- 0.3 Conventions
- 0.4 Time budget
- 0.5 Traceability: which phase answers which question

Phase 0. Baseline Recovery and Evidence Capture

- 0.1 Preserve the Round 1 evidence
- 0.2 Rotate the committed private key
- 0.3 Apply the schema migrations
- 0.4 Clean start and a 30 minute reference run
- 0.5 Gate
- 0.6 Capture the baseline numbers
- 0.7 Baseline reference table

Phase 1. Datasets and Multi-Camera Topology

- 1.1 ReID evaluation dataset
- 1.2 Multi-camera simulator with real identity overlap
- 1.3 Ground-truth annotation for the topology
- 1.4 PS-4 object datasets
- 1.5 Register the topology and gate

Phase 2. ReID Model Export and Offline Evaluation

- 2.1 Acquire and export OSNet
- 2.2 Quantize to INT8 with a real calibration set
- 2.3 Offline evaluation on Market-1501
- 2.4 Face recognition comparison, so the answer is measured
- 2.5 Gate and results table

Phase 3. Edge Embedding Integration

- 3.1 Wire the real model in
- 3.2 Verify the embedding contract end to end
- 3.3 Confirm one embedding per tracklet, not per frame
- 3.4 Measure the wire payload
- 3.5 Gate

Phase 4. Cloud Identity Plane and Threshold Calibration

- 4.1 Enable pgvector and migrate the embedding column
- 4.2 The resolver service
- 4.3 Recompute centroids on a schedule (Q3.1d, drift)
- 4.4 The calibration sweep: this is the data-driven core
- 4.5 Cross-camera evaluation on your own topology
- 4.6 Domain shift measurement (Q3.1d)

Phase 5. Gallery Scale Benchmark, 1K to 1M

- 5.1 Synthetic gallery generator
- 5.2 Flat versus HNSW versus IVFFLAT
- 5.3 The same question inside Postgres
- 5.4 Storage cost model (Q3.1e)
- 5.5 Results table and gate

Phase 6. Privacy, Consent, Retention, Audit

- 6.1 Seed the consent model
- 6.2 The DSAR endpoint (Q3.2b)
- 6.3 The erasure drill

- 6.4 Retention TTL under time compression

Phase 7. False Positive Remediation Drill

- 7.1 Reproduce the failure deliberately
- 7.2 Diagnose it the way you would in a support ticket (Q3.3a)
- 7.3 The immediate fix, and measure how fast it lands (Q3.3b)
- 7.4 The systemic change, measured (Q3.3c)

Phase 8. PS-4 Detection Architecture Benchmark

- 8.1 Build the class inventory
- 8.2 Strategy A versus Strategy B, benchmarked
- 8.3 The scaling argument, extrapolated from the measurement

Phase 9. New Customer Class in Under One Day

- 9.1 Start the clock
- 9.2 Build the prototype gallery from a handful of examples
- 9.3 Ship it to the edge through the control plane
- 9.4 Evaluate the new class without a single training step
- 9.5 Stop the clock and write the timeline

Phase 10. Model Lifecycle: Canary, Cohorts, Per-Tenant Rollback

- 10.1 Model registry
- 10.2 Manufacture a regression, on purpose
- 10.3 Feed the cohort metrics table
- 10.4 Run the canary gate
- 10.5 The split rollback drill

Phase 11. Inference Cost Reduction to 50 Percent

- 11.1 Benchmark every model variant on one bench
- 11.2 Quantify each lever separately
- 11.3 Temporal caching, measured (Q4.3c)
- 11.4 Unit economics

Phase 12. Chaos, Partition, Domain Shift

- 12.1 Toxiproxy in front of the cloud
- 12.2 The partition drill
- 12.3 Degradation modes, not just outage
- 12.4 Domain shift, quantified

Phase 13. Observability

- 13.1 Prometheus exporter over the JSONL telemetry
- 13.2 Prometheus and Grafana
- 13.3 The dashboard panels, mapped to questions

Phase 14. Results Extraction and Deliverables

- 14.1 The consolidator
- 14.2 Regenerate the headline table
- 14.3 Final full-system run

Appendix A. Command Reference

- Lifecycle
- Inspection
- Emitting directives by hand

Appendix B. Environment Variables

Appendix C. Troubleshooting

Appendix D. What to Say, and What Not To

Part 0. Ground Rules

0.1 What this document is

This is the build plan for Round 2, covering PS-3 (Person Recognition and Identity Intelligence) and PS-4 (Object and Asset Recognition), executed as an extension of the existing `video-intel` prototype rather than as a new project.

It is written to be followed literally. Every phase states its inputs, the exact commands, the file it produces, and a **gate** that must pass before the next phase begins. If a gate fails, stop and fix it; do not carry a broken stage forward, because every later measurement inherits its errors.

Three principles govern everything below.

Measure before you claim. No number enters a deliverable unless a script in `tools/` produced it from a log file in `data/logs/`. Round 1 shipped claims that came from a decode bug (`n_det == 10` in 3,211 of 3,211 records), and the only defence against repeating that is to make every number reproducible by command.

Assert the invariant, not just the metric. A metric that can read 233%, as the Round 1 completeness SLI did in 168 of 168 samples, can also read 100% while a camera is dead. Every SLI added below carries a bounds check enforced by `tools/verify.py` in CI.

Reuse the primitive, do not rebuild it. The outbox, the idempotency key, and the edge-first inference placement already exist and already solve the hard parts of PS-3 and PS-4. Seven Round 2 requirements collapse into one versioned control channel. Building them separately is how a codebase rots.

0.2 Prerequisite state

Before Phase 0, the repository must be at the post-patch state described in `ROUND2_PLAN.md`, with these files present.

Path	Role
<code>edge/config.py</code>	path and tunable resolution, no hardcoded paths
<code>edge/detect.py</code>	shape-resolved ONNX decode, letterbox, ImageNet norm
<code>edge/infer_pool.py</code>	one shared session, bounded queue, drop counter
<code>edge/tracker.py</code>	IoU-gated association, temporal confirmation
<code>edge/zones.py</code>	dwelling, tripwire, crowd
<code>edge/embed.py</code>	per-tracklet descriptor
<code>edge/inbox.py</code>	control plane, seven appliers
<code>edge/policy.py</code>	per-tenant versioned flags
<code>edge/retention.py</code>	TTL, disk pressure, consent purge
<code>edge/schema.sql</code>	SQLite, 11 tables
<code>cloud/api.py</code>	pooled ingest, <code>/control</code> , <code>/health/fleet</code>
<code>cloud/identity.py</code>	gallery, margin test, three-way resolve
<code>cloud/adjudicate.py</code>	review queue, canary gate
<code>cloud/schema.sql</code>	Postgres, 13 tables
<code>tools/verify.py</code>	invariant harness

Verify placement before starting. The two `schema.sql` files are not interchangeable and a swap fails at agent boot.

```
head -1 edge/schema.sql # must read: -- Edge schema (SQLite)
head -1 cloud/schema.sql # must read: -- Cloud schema (Postgres)
python -c "import edge.config, edge.detect, edge.inbox, edge.policy; print('edge OK')"
python -c "import cloud.identity, cloud.adjudicate; print('cloud OK')"
```

0.3 Conventions

Branching. One branch per phase, merged only after its gate passes. This gives you a bisectable history and a story to tell about how the system was built.

```
git checkout -b phase/NN-short-name
# ... work ...
python tools/verify.py data/logs/*.jsonl --strict && git commit -am "phase NN: <gate passed>"
git checkout main && git merge --no-ff phase/NN-short-name
```

Directory contract. Anything under `results/` is generated and is the only thing a deliverable may cite. Anything under `data/` is input or raw telemetry. Never hand-edit `results/`.

```
mkdir -p results/{baseline,reid,identity,scene,privacy,objects,lifecycle,cost,chaos}
mkdir -p tools/bench data/datasets data/models/registry
echo "results/" >> .gitignore # regenerate, do not version
```

Environment. A single `.env` sourced by every shell. No absolute paths in code, ever.

```
cat > .env <<'EOF'
export VI_ROOT="$PWD"
export DATA="$PWD/data"
export PYTHONPATH="$PWD"
export DATABASE_URL="postgresql://user:pass@host.neon.tech/videointel?sslmode=require"
export VI_API_TOKEN="$(openssl rand -hex 24)"
export CLOUD_API="http://127.0.0.1:8000"
```

```
export VI_TENANT="default"
export VI_TARGET_FPS=4
export VI_CONF_HIGH=0.50
export VI_CONF_LOW=0.30
export VI_RETAIN_DAYS=30
EOF
echo ".env" >> .gitignore
source .env
```

Definition of done. A phase is done when: its gate command exits 0, its artefact exists under `results/`, and `tools/verify.py --strict` still exits 0 on the accumulated logs. Not when the code looks finished.

0.4 Time budget

Phase	Title	Hours	Cumulative
0	Baseline recovery and evidence capture	2.0	2.0
1	Datasets and multi-camera topology	3.0	5.0
2	ReID model export and offline evaluation	3.0	8.0
3	Edge embedding integration	2.5	10.5
4	Cloud identity plane and threshold calibration	3.5	14.0
5	Gallery scale benchmark, 1K to 1M	2.5	16.5
6	Privacy, consent, retention, audit	2.5	19.0
7	False positive remediation drill	1.5	20.5
8	PS-4 detection architecture benchmark	3.0	23.5
9	PS-4 new class in under one day	2.0	25.5
10	Model lifecycle, canary, per-tenant rollback	3.0	28.5
11	Inference cost reduction to 50 percent	3.0	31.5
12	Chaos, partition, domain shift	2.5	34.0
13	Observability	1.5	35.5
14	Results extraction and deliverables	2.5	38.0

If time is short, the minimum viable path is Phases 0, 2, 4, 8, 10, 14. Those six produce evidence for every question. Everything else deepens it.

0.5 Traceability: which phase answers which question

Keep this table open while working. If a phase produces no row here, it is not earning its time.

Question	Topic	Phases	Artefact
Q3.1a	Face recognition vs ReID	2, 4	results/reid/eval_market1501.json
Q3.1b	1K to 1M database scaling	5	results/scale/scaling_curve.csv
Q3.1c	When a vector DB is necessary	5	results/scale/index_comparison.md
Q3.1d	Embedding drift over time	4, 10	results/identity/drift_sim.json
Q3.1e	Storage cost per identity	5	results/scale/storage_model.csv
Q3.2a	Consent at scale	6	results/privacy/consent_flow.md
Q3.2b	Audit trail design	6	results/privacy/audit_sample.json
Q3.2c	Time-based deletion	6	results/privacy/deletion_receipt.json
Q3.2d	Hard problems in deletion	6	narrative, grounded in 6.4
Q3.3a	Diagnosing a false match	7	results/privacy/fp_diagnosis.json
Q3.3b	Immediate fix	7	results/privacy/exclusion_latency.json
Q3.3c	Systemic change	7, 10	results/lifecycle/margin_impact.csv
Q4.1a	Monolithic vs modular	8	results/objects/arch_benchmark.csv
Q4.1b	Classification vs embedding retrieval	8, 9	results/objects/openset_eval.json
Q4.1c	New customer class in under a day	9	results/objects/new_class_timeline.md
Q4.1d	Strategy A vs B	8	results/objects/arch_benchmark.csv
Q4.2a	Catching a bad deploy earlier	0, 10	results/lifecycle/canary_log.json
Q4.2b	Per-customer rollback	10	results/lifecycle/rollback_drill.json
Q4.2c	A/B testing in production	10	results/lifecycle/ab_design.md
Q4.2d	Cohort-scoped metrics	10	results/lifecycle/cohort_metrics.csv
Q4.3a	Quantization, pruning, distillation	11	results/cost/model_variants.csv
Q4.3b	Multi-stage inference	11	results/cost/cascade_model.csv
Q4.3c	Caching strategy	11	results/cost/cache_eval.json
Q4.3d	50 percent cost reduction	11	results/cost/reduction_summary.md

Phase 0. Baseline Recovery and Evidence Capture

Budget 2.0 h | **Branch** `phase/00-baseline` | **Gate** `tools/verify.py --strict` exits 0 on a fresh 30 minute run

The purpose of this phase is not to build anything. It is to establish a trustworthy "before" measurement, because every claim in the Round 2 deliverable is a delta against it, and a delta against a corrupted baseline is worthless.

0.1 Preserve the Round 1 evidence

The pre-fix logs are the proof that the bugs existed. They are the raw material for the incident narrative. Archive them before anything overwrites them.

```
source .env
mkdir -p results/baseline/round1_raw
cp data/logs/edge_a.jsonl results/baseline/round1_raw/
cp data/logs/edge_b.jsonl results/baseline/round1_raw/
cp data/logs/mediamtx.log results/baseline/round1_raw/
cp data/edge_a.db results/baseline/round1_raw/
sha256sum results/baseline/round1_raw/* > results/baseline/round1_raw/SHA256SUMS
```

Run the harness against the archived logs and save the failure output verbatim. This file is a deliverable in its own right.

```
python tools/verify.py results/baseline/round1_raw/edge_a.jsonl \
2>&1 | tee results/baseline/round1_verify_FAIL.txt
```

Expected content, which is the evidence for Q4.2a:

```
FAIL detect_cap_not_always_hit
cap hit on 100.0% of 3211 detections.
FAIL detect_count_has_variance
1 distinct detection count across 3211 frames (mode=(10, 3211)).
FAIL completeness_within_bounds
168/168 readings outside [0, 1.05], max=2.33.
WARN inference_keeps_up
p95 detect latency 1933 ms against a 250 ms budget at 4 fps.
```

0.2 Rotate the committed private key

`auto.key` and `auto.crt` are in the public repository history. Rotating the key is necessary but not sufficient; the history must be purged, or the old key is still retrievable by anyone who clones.

```
git rm --cached auto.key auto.crt
printf 'auto.key\nauto.crt\n*.pem\n*.key\n.env\n' >> .gitignore
git commit -m "remove committed TLS material"

pip install git-filter-repo --break-system-packages
git filter-repo --path auto.key --path auto.crt --invert-paths --force
git push origin --force --all

openssl req -x509 -newkey rsa:2048 -nodes -days 365 \
-keyout auto.key -out auto.crt -subj "/CN=video-intel-local"
```

0.3 Apply the schema migrations

Both files are additive and idempotent, so they are safe against the existing databases. Run them and confirm the table counts.

```
sqlite3 data/edge_a.db < edge/schema.sql
sqlite3 data/edge_b.db < edge/schema.sql
psql "$DATABASE_URL" -f cloud/schema.sql

sqlite3 data/edge_a.db "SELECT COUNT(*) AS edge_tables FROM sqlite_master WHERE type='table';"
psql "$DATABASE_URL" -c "SELECT COUNT(*) AS cloud_tables FROM information_schema.tables WHERE table_schema='public';"
```

Expect 11 edge tables and 13 cloud tables. If the edge count is lower, the Postgres schema was applied to SQLite by mistake and the `CREATE EXTENSION` line aborted the script.

0.4 Clean start and a 30 minute reference run

```
kill -f "python -m edge.agent"; kill -f ffmpeg; kill -f mediamtx; kill -f uvicorn
sleep 2
rm -f data/logs/edge_*.jsonl
rm -rf data/seg/* && mkdir -p data/seg

nohup ops/run_mediatrix.sh > data/logs/mediamtx.log 2>&1 &
sleep 3
sim/spawn.sh
sleep 5
nohup uvicorn cloud.api:app --host 0.0.0.0 --port 8000 > data/logs/api.log 2>&1 &
sleep 3
edge/run.sh a & edge/run.sh b &

echo "reference run started $(date -Iseconds)" | tee results/baseline/run_start.txt
sleep 1800
```

0.5 Gate

```
python tools/verify.py data/logs/edge_a.jsonl data/logs/edge_b.jsonl --strict \
2>&1 | tee results/baseline/round2_verify_PASS.txt
echo "exit=$?"
```

GATE 0

Exit code 0. Both sites present in the log. `detect_count_has_variance` passes with at least three distinct values. `completeness_within_bounds` passes. `all_configured_cameras_report` shows every camera in `roster.yaml` producing detections, which is the check that would have caught the empty `edge_b.db` in Round 1.

0.6 Capture the baseline numbers

Write the extractor once and reuse it in every later phase.

```
cat > tools/baseline.py <<'PY'
#!/usr/bin/env python3
"""Extract the reference metrics every later phase compares against."""
import json, sys, statistics, collections, pathlib

def load(paths):
    rec = collections.defaultdict(list)
    for p in paths:
        for line in open(p):
            try: d = json.loads(line)
            except Exception: continue
            rec[d.get("stage", "?")].append(d)
    return rec

def pct(v, q):
    v = sorted(v); return v[min(len(v)-1, int(q*len(v)))] if v else 0.0

r = load(sys.argv[1:])
lat = [x["latency_ms"] for x in r["detect"] if "latency_ms" in x]
g = r["motion_gate"]
gp = sum(1 for x in g if x.get("passed"))
counts = [x.get("n_confident", 0) for x in r["detect_result"]]
pool = r["infer_pool"]
sub = sum(x.get("submitted", 0) for x in pool); drop = sum(x.get("dropped", 0) for x in pool)

out = {
    "detect_n": len(lat),
    "detect_mean_ms": round(statistics.mean(lat), 1) if lat else None,
    "detect_p50_ms": round(pct(lat, .50), 1),
    "detect_p95_ms": round(pct(lat, .95), 1),
    "detect_p99_ms": round(pct(lat, .99), 1),
    "gate_evaluated": len(g),
    "gate_pass_rate": round(gp/len(g), 4) if g else None,
    "detections_per_frame_mean": round(statistics.mean(counts), 2) if counts else 0,
    "detections_distinct_values": len(set(counts)),
    "infer_submitted": sub, "infer_dropped": drop,
    "infer_drop_rate": round(drop/sub, 4) if sub else 0,
    "tracks_confirmed": len(r["track_confirmed"]),
    "activity_events": len(r["activity"]),
    "outbox_peak_depth": max([x.get("depth", 0) for x in r["outbox"]] or [0]),
    "outbox_final_depth": r["outbox"][-1].get("depth") if r["outbox"] else None,
    "outbox_dead_letters": r["outbox"][-1].get("dlq") if r["outbox"] else None,
    "completeness_min": min([x.get("ratio", 1) for x in r["completeness"]] or [1]),
    "completeness_max": max([x.get("ratio", 0) for x in r["completeness"]] or [0]),
    "completeness_faults": len(r["completeness_fault"]),
    "thread_crashes": len(r["thread_crash"]),
}
pathlib.Path("results/baseline/metrics.json").write_text(json.dumps(out, indent=2))
print(json.dumps(out, indent=2))
PY
chmod +x tools/baseline.py
python tools/baseline.py data/logs/edge_a.jsonl data/logs/edge_b.jsonl
```

0.7 Baseline reference table

Fill this from `results/baseline/metrics.json`. The Round 1 column is from the archived logs and is the "before" you will cite.

Metric	Round 1 (buggy)	Round 2 baseline	Source
Detections per frame, distinct values	1	fill in	<code>detections_distinct_values</code>
Detect p95 latency	1933 ms	fill in	<code>detect_p95_ms</code>
Gate pass rate	0.814	fill in	<code>gate_pass_rate</code>
Inference drop rate	not measured	fill in	<code>infer_drop_rate</code>
Completeness range	1.67 to 2.33	fill in	<code>completeness_min/max</code>
Outbox peak depth	934	fill in	<code>outbox_peak_depth</code>
Dead letters	not tracked	fill in	<code>outbox_dead_letters</code>
Activity events	0, unimplemented	fill in	<code>activity_events</code>

DO NOT SKIP THIS

Every percentage in the final deliverable is a ratio against this table. If you start Phase 1 without `results/baseline/metrics.json` on disk, you will end up estimating deltas at the end, and estimated deltas are exactly what Round 1 was penalised for.

Phase 1. Datasets and Multi-Camera Topology

Budget 3.0 h | **Branch** `phase/01-data` | **Gate** a person visibly walks from `a_cam01` into `a_cam02` with ground-truth identity labels

PS-3 cannot be evaluated on the Round 1 simulator. Twelve independent looping clips have no shared identities, so cross-camera re-identification has nothing to re-identify. This phase builds a topology where the same person appears in multiple cameras with known labels, and acquires the datasets that let you report Rank-1 and mAP rather than adjectives.

1.1 ReID evaluation dataset

Market-1501 is the standard benchmark: 1,501 identities, 6 cameras, with a defined query and gallery split. It is what lets you say "Rank-1 of 0.87" with a number a reviewer can check.

```
mkdir -p data/datasets && cd data/datasets

# Market-1501 (approx 145 MB). Mirrors move; if this 404s, search
# "Market-1501-v15.09.15" on Kaggle or the OpenDataLab mirror.
pip install kaggle --break-system-packages
kaggle datasets download -d pengcwl/market-1501 -p . --unzip

ls Market-1501-v15.09.15/
# bounding_box_train/ bounding_box_test/ query/ gt_bbox/
cd "$VI_ROOT"
```

Filename encoding matters, because it is your label source: `0002_c1s1_000451_03.jpg` means identity `0002`, camera `c1`, sequence `s1`, frame `000451`, box `03`. Identity `-1` is a distractor and must be excluded from evaluation.

```
python - <<'PY'
import pathlib, collections
root = pathlib.Path("data/datasets/Market-1501-v15.09.15")
for split in ("bounding_box_train", "query", "bounding_box_test"):
    f = list((root/split).glob("*.jpg"))
    ids = {p.name.split("_")[0] for p in f}
    cams = {p.name.split("_")[1][1:2] for p in f}
    print(f"{split:22s} {len(f):6d} imgs {len(ids):5d} ids {sorted(cams)}")
PY
```

Expect roughly 12,936 train / 3,368 query / 19,732 gallery images.

1.2 Multi-camera simulator with real identity overlap

The trick is to build a topology where the *same source video* feeds two cameras with a time offset and different crops. A person leaving the right edge of `a_cam01` then enters the left edge of `a_cam02` twelve seconds later, which is exactly the cross-camera handoff PS-3 has to solve, and you know the ground truth because you built it.

```
cat > sim/build_topology.sh <<'SH'
#!/usr/bin/env bash
# Build a 4-camera site where cameras share identities via time-offset crops.
set -euo pipefail
SRC="${1:?usage: build_topology.sh <source.mp4>}"
OUT="$DATA/sim"; mkdir -p "$OUT"

# a_cam01: right half of frame, t=0 (person exits right)
ffmpeg -y -i "$SRC" -vf "crop=iw/2:ih:iw/2:0,scale=640:360" \
  -c:v libx264 -preset veryfast -b:v 900k -g 50 -an "$OUT/a_cam01.mp4"

# a_cam02: left half, delayed 12s (same person enters left)
ffmpeg -y -ss 12 -i "$SRC" -vf "crop=iw/2:ih:0:0,scale=640:360" \
  -c:v libx264 -preset veryfast -b:v 900k -g 50 -an "$OUT/a_cam02.mp4"

# a_cam03: full frame, delayed 30s, dimmed (illumination domain shift)
ffmpeg -y -ss 30 -i "$SRC" -vf "scale=640:360,eq=brightness=-0.12:contrast=1.15" \
  -c:v libx264 -preset veryfast -b:v 900k -g 50 -an "$OUT/a_cam03.mp4"

# a_cam04: full frame, delayed 45s, downsampled then upsampled (resolution shift)
ffmpeg -y -ss 45 -i "$SRC" -vf "scale=320:180,scale=640:360" \
  -c:v libx264 -preset veryfast -b:v 900k -g 50 -an "$OUT/a_cam04.mp4"

for f in "$OUT"/a_cam0*.mp4; do
    printf "%-24s %s\n" "$(basename "$f")" "$(ffprobe -v error \
      -show_entries format=duration -of csv=p=0 "$f")s"
done
SH
chmod +x sim/build_topology.sh
sim/build_topology.sh data/datasets/shanghaitech/source.mp4
```

Cameras 3 and 4 exist to create *controlled* domain shift: one illumination, one resolution. In Phase 4 you will measure how much each costs you in Rank-1, which is the empirical answer to Q3.1d that most candidates only assert.

1.3 Ground-truth annotation for the topology

Cross-camera evaluation needs labels. Annotating by hand is slow; annotate by construction instead. Because `a_cam02` is `a_cam01` offset by exactly 12 seconds, a track in cam02 at time `t` is the same person as a track in cam01 at `t - 12` if their appearance windows overlap.

```
cat > tools/build_gt.py <<'PY'
#!/usr/bin/env python3
"""Derive cross-camera ground truth from the known offsets in the topology.
Two tracklets are the same identity when their time windows align under the
```



```

camera offset and their spatial handoff is consistent."""
import json, sys, collections, itertools

OFFSETS = {"a_cam01": 0.0, "a_cam02": 12.0, "a_cam03": 30.0, "a_cam04": 45.0}
TOL = 2.5 # seconds of slack for tracker start/stop jitter

tracks = collections.defaultdict(list)
for line in open(sys.argv[1]):
    d = json.loads(line)
    if d.get("stage") != "track_end":
        continue
    cam = d["camera_id"]
    tracks[cam].append({"track_id": d["track_id"],
                        "start": d["first_ts"] - OFFSETS.get(cam, 0),
                        "end": d["last_ts"] - OFFSETS.get(cam, 0)})

pairs, next_gid, gid = [], 1, {}
for (c1, t1), (c2, t2) in itertools.combinations(tracks.items(), 2):
    for a in t1:
        for b in t2:
            if abs(a["start"] - b["start"]) < TOL and abs(a["end"] - b["end"]) < TOL:
                k1, k2 = (c1, a["track_id"]), (c2, b["track_id"])
                g = gid.get(k1) or gid.get(k2) or next_gid
                if g == next_gid: next_gid += 1
                gid[k1] = gid[k2] = g
                pairs.append({"a": k1, "b": k2, "gid": g})

json.dump({"pairs": [{"cam_a": p["a"][0], "track_a": p["a"][1],
                    "cam_b": p["b"][0], "track_b": p["b"][1],
                    "gid": p["gid"]} for p in pairs],
          "n_identities": next_gid - 1},
          open("results/identity/ground_truth.json", "w"), indent=2)
print(f"{len(pairs)} cross-camera pairs, {next_gid-1} identities")
PY
chmod +x tools/build_gt.py

```

1.4 PS-4 object datasets

PS-4 needs a class inventory that grows. Roboflow Universe is the fastest source of small, well-labelled industrial datasets in YOLO format.

```

pip install roboflow supervision ultralytics --break-system-packages
mkdir -p data/datasets/objects && cd data/datasets/objects

python - <<'PY'
from roboflow import Roboflow
import os
rf = Roboflow(api_key=os.environ["ROBOFLOW_API_KEY"])
# Base inventory: PPE compliance, the canonical 20-class starting point
rf.workspace("roboflow-universe-projects").project("construction-site-safety") \
    .version(30).download("yolov8", location="ppe")
# The "new customer class" for Phase 9. Held out deliberately.
rf.workspace("roboflow-100").project("forklift-rzcx1") \
    .version(2).download("yolov8", location="forklift")
PY

for d in ppe forklift; do
    echo "=== $d ==="; cat $d/data.yaml | head -20
    echo 'images: $(find $d -name '*.jpg' | wc -l) '
done
cd "$VI_ROOT"

```

WHY FORKLIFT IS HELD OUT

Phase 9 must demonstrate introducing a class the base model has never seen, in under one working day, without retraining. Downloading it now and refusing to touch it until Phase 9 is what makes that demonstration honest rather than staged.

1.5 Register the topology and gate

```

cat > edge/roster.yaml <<'YML'
sites:
  a: {tenant: acme-retail, cameras: [a_cam01, a_cam02, a_cam03, a_cam04]}
  b: {tenant: globex-wh, cameras: [b_cam01, b_cam02]}
analyse: [a_cam01, a_cam02, a_cam03, a_cam04, b_cam01, b_cam02]
record_all: true
reid_enabled: [a_cam01, a_cam02, a_cam03, a_cam04]
YML

sim/spawn.sh
ffprobe -v error -i rtsp://127.0.0.1:8554/a_cam01 -show_streams 2>&1 | head -8
ffprobe -v error -i rtsp://127.0.0.1:8554/a_cam02 -show_streams 2>&1 | head -8

```

GATE 1

Four site-A streams playable over RTSP. <data/datasets/Market-1501-v15.09.15/query> contains 3,368 images. Both object datasets present with a [data.yaml](#). A 10 minute agent run followed by [tools/build_gt.py](#) reports at least 20 cross-camera pairs.

Phase 2. ReID Model Export and Offline Evaluation

Budget 3.0 h | **Branch** `phase/02-reid` | **Gate** INT8 ONNX model reaching Rank-1 above 0.80 on Market-1501 at under 15 ms per crop on CPU

This phase replaces the placeholder descriptor in `edge/embed.py` with a real model and, more importantly, produces the numbers that answer Q3.1a. The face-versus-ReID argument is unwinnable without measurements: face recognition needs roughly 60 pixels of interocular distance, and surveillance cameras at typical mounting heights rarely deliver it. You will show that from your own footage rather than asserting it.

2.1 Acquire and export OSNet

OSNet-x0.25 is the right size for CPU edge inference: 0.6 M parameters, 512-dimension output, and Rank-1 near 0.85 on Market-1501 at a fraction of the cost of a ResNet-50 backbone.

```
pip install torch torchvision torchreid onnx onnxsim onnxruntime \
--break-system-packages

mkdir -p data/models/registry
python - <<'PY'
import torch, torchreid
model = torchreid.models.build_model(
    name="osnet_x0_25", num_classes=751, loss="softmax", pretrained=True)
model.eval()
# Strip the classifier: we want embeddings, not class logits
model.classifier = torch.nn.Identity()

dummy = torch.randn(1, 3, 256, 128)          # ReID convention: H=256, W=128
torch.onnx.export(
    model, dummy, "data/models/registry/reid-osnet-x025-fp32.onnx",
    input_names=["images"], output_names=["embedding"],
    dynamic_axes={"images": {0: "batch"}, "embedding": {0: "batch"}},
    opset_version=13, do_constant_folding=True)
print("exported, embedding dim:", model(dummy).shape)
PY

python -m onnxsim data/models/registry/reid-osnet-x025-fp32.onnx \
    data/models/registry/reid-osnet-x025-fp32.onnx
```

Verify the graph signature before trusting it. This is the habit that would have prevented the Round 1 detector bug.

```
python - <<'PY'
import onnx
m = onnx.load("data/models/registry/reid-osnet-x025-fp32.onnx")
for i in m.graph.input:
    print("IN ", i.name, [d.dim_value or d.dim_param for d in i.type.tensor_type.shape.dim])
for o in m.graph.output:
    print("OUT", o.name, [d.dim_value or d.dim_param for d in o.type.tensor_type.shape.dim])
PY
```

Expect `IN images [batch, 3, 256, 128]` and `OUT embedding [batch, 512]`.

2.2 Quantize to INT8 with a real calibration set

Dynamic quantization is one line but leaves accuracy on the table for convolutional backbones. Static quantization with calibration data from your *deployment* distribution, not from Market-1501, is what keeps edge accuracy close to fp32.

```
cat > tools/quantize_reid.py <<'PY'
#!/usr/bin/env python3
import glob, numpy as np, cv2
from onnxruntime.quantization import (
    quantize_static, CalibrationDataReader, QuantType, QuantFormat)

MEAN = np.array([0.485, 0.456, 0.406], np.float32)
STD = np.array([0.229, 0.224, 0.225], np.float32)

def prep(path):
    img = cv2.imread(path)
    img = cv2.resize(img, (128, 256))[:, :, ::-1].astype(np.float32) / 255.0
    img = (img - MEAN) / STD
    return np.ascontiguousarray(img.transpose(2, 0, 1)[None])

class Reader(CalibrationDataReader):
    def __init__(self, files):
        self.it = iter([{"images": prep(f)} for f in files])
    def get_next(self):
        return next(self.it, None)

# Calibrate on crops from YOUR cameras, not the benchmark. 300 is plenty.
files = sorted(glob.glob("data/calib/crops/*.jpg"))[:300]
assert len(files) >= 100, f"need >=100 calibration crops, found {len(files)}"

quantize_static(
    "data/models/registry/reid-osnet-x025-fp32.onnx",
    "data/models/registry/reid-osnet-x025-int8.onnx",
    Reader(files),
    quant_format=QuantFormat.QDQ,
    activation_type=QuantType.QUInt8,
    weight_type=QuantType.QInt8,
```

```

    per_channel=True)
print("quantized")
PY
chmod +x tools/quantize_reid.py

```

Harvest the calibration crops from a short live run first.

```

mkdir -p data/calib/crops
cat > tools/harvest_crops.py <<'PY'
#!/usr/bin/env python3
"""Dump person crops from the live simulator for quantization calibration."""
import os, cv2, itertools
os.environ.setdefault("VI_ROOT", os.getcwd())
from edge import detect
d = detect.Detector(classes={1})
n = 0
for cam in ("a_cam01", "a_cam02", "a_cam03", "a_cam04"):
    cap = cv2.VideoCapture(f"data/sim/{cam}.mp4")
    for i in itertools.count():
        ok, f = cap.read()
        if not ok or n >= 400: break
        if i % 15: continue
        conf, _ = d(f, cam)
        for det in conf:
            x1, y1, x2, y2 = [int(v) for v in det.box]
            crop = f[max(0,y1):y2, max(0,x1):x2]
            if crop.size and (x2-x1) > 25 and (y2-y1) > 55:
                cv2.imwrite(f"data/calib/crops/{cam}_{n:04d}.jpg", crop); n += 1
print("crops:", n)
PY
python tools/harvest_crops.py
python tools/quantize_reid.py
ls -la data/models/registry/

```

Record the size reduction; it is a data point for Q4.3a.

```

python - <<'PY'
import os
for f in ("fp32", "int8"):
    p = f"data/models/registry/reid-osnet-x025-{f}.onnx"
    print(f"{f}: {os.path.getsize(p)/1e6:.1f} MB")
PY

```

2.3 Offline evaluation on Market-1501

This script produces the headline ReID numbers. Run it for both fp32 and int8.

```

cat > tools/eval_reid.py <<'PY'
#!/usr/bin/env python3
"""Rank-1 / Rank-5 / mAP on Market-1501 with the standard protocol:
a gallery item is excluded when it shares BOTH identity and camera with the
query, because matching the same person in the same camera is trivial."""
import sys, glob, json, time, pathlib
import numpy as np, cv2, onnxruntime as ort

MEAN = np.array([0.485,0.456,0.406], np.float32)
STD = np.array([0.229,0.224,0.225], np.float32)
ROOT = pathlib.Path("data/datasets/Market-1501-v15.09.15")

def meta(p):
    n = pathlib.Path(p).name.split("_")
    return int(n[0]), int(n[1][1]) # pid, camid

def embed_all(sess, files, bs=32):
    inp = sess.get_inputs()[0].name
    out, lat = [], []
    for i in range(0, len(files), bs):
        batch = []
        for f in files[i:i+bs]:
            img = cv2.imread(f)
            img = cv2.resize(img, (128,256))[:,::-1].astype(np.float32)/255.0
            batch.append(((img-MEAN)/STD).transpose(2,0,1))
        x = np.ascontiguousarray(np.stack(batch), dtype=np.float32)
        t0 = time.perf_counter()
        v = sess.run(None, {inp: x})[0]
        lat.append((time.perf_counter()-t0)*1000/len(batch))
        out.append(v / (np.linalg.norm(v, axis=1, keepdims=True) + 1e-12))
    return np.vstack(out), float(np.mean(lat))

model = sys.argv[1]
so = ort.SessionOptions(); so.intra_op_num_threads = 2
so.graph_optimization_level = ort.GraphOptimizationLevel.ORT_ENABLE_ALL
sess = ort.InferenceSession(model, so, providers=["CPUExecutionProvider"])

q = sorted(glob.glob(str(ROOT/"query"/*.jpg)))
g = sorted(glob.glob(str(ROOT/"bounding_box_test"/*.jpg)))
    if not pathlib.Path(f).name.startswith("-1")
print(f"query {len(q)} gallery {len(g)}")

Q, q_ms = embed_all(sess, q); G, g_ms = embed_all(sess, g)
qm = np.array([meta(f) for f in q]); gm = np.array([meta(f) for f in g])

r1 = r5 = 0; aps = []; margins = []
for i in range(len(q)):
    sims = G @ Q[i]
    valid = ~((gm[:,0] == qm[i,0]) & (gm[:,1] == qm[i,1]))
    s, gp = sims[valid], gm[valid,0]
    order = np.argsort(-s); ranked = gp[order]

```

```

hit = ranked == qm[i,0]
if not hit.any(): continue
r1 += hit[0]; r5 += hit[:5].any()
margins.append(float(s[order][0] - s[order][1]))
cum = np.cumsum(hit)
aps.append(float(np.sum(cum[hit] / (np.where(hit)[0] + 1)) / hit.sum()))

res = {"model": model,
      "rank1": round(r1/len(q), 4), "rank5": round(r5/len(q), 4),
      "mAP": round(float(np.mean(aps)), 4),
      "margin_mean": round(float(np.mean(margins)), 4),
      "margin_p05": round(float(np.percentile(margins, 5)), 4),
      "ms_per_crop_query": round(q_ms, 2),
      "ms_per_crop_gallery": round(g_ms, 2),
      "dim": int(Q.shape[1]), "n_query": len(q), "n_gallery": len(g)}
name = pathlib.Path(model).stem
pathlib.Path("results/reid").mkdir(parents=True, exist_ok=True)
pathlib.Path(f"results/reid/eval_{name}.json").write_text(json.dumps(res, indent=2))
print(json.dumps(res, indent=2))
PY
chmod +x tools/eval_reid.py

python tools/eval_reid.py data/models/registry/reid-osnet-x025-fp32.onnx
python tools/eval_reid.py data/models/registry/reid-osnet-x025-int8.onnx

```

MARGIN_P05 IS THE NUMBER TO REMEMBER

It is the 5th percentile of the gap between the best and second-best gallery match. It tells you directly what **delta** to set in the margin test in Phase 4, and it is the quantitative reason the John/David false match happens: if the 5th percentile margin is 0.03, then one query in twenty is effectively a coin flip between two identities, and a system with a single threshold will call all of them confidently.

2.4 Face recognition comparison, so the answer is measured

Q3.1a asks you to choose between face recognition and ReID. Measure the reason.

```

pip install insightface --break-system-packages
cat > tools/face_feasibility.py <<'PY'
#!/usr/bin/env python3
"""What fraction of detected persons in OUR footage carry a face large enough
for recognition? Industry guidance is ~60 px interocular for reliable 1:N."""
import os, cv2, json, itertools, numpy as np, pathlib
os.environ.setdefault("VI_ROOT", os.getcwd())
from edge import detect
from insightface.app import FaceAnalysis

fa = FaceAnalysis(name="buffalo_s", providers=["CPUExecutionProvider"])
fa.prepare(ctx_id=-1, det_size=(640, 640))
d = detect.Detector(classes={1})

persons = faces = usable = 0; eyedists = []
for cam in ("a_cam01", "a_cam02", "a_cam03", "a_cam04"):
    cap = cv2.VideoCapture(f"data/sim/{cam}.mp4")
    for i in itertools.count():
        ok, f = cap.read()
        if not ok or i > 900: break
        if i % 25: continue
        conf, _ = d(f, cam)
        persons += len(conf)
        for det in conf:
            x1,y1,x2,y2 = [int(v) for v in det.box]
            crop = f[max(0,y1):y2, max(0,x1):x2]
            if crop.size == 0: continue
            for fc in fa.get(crop):
                faces += 1
                kp = fc.kps
                ed = float(np.linalg.norm(kp[0] - kp[1]))
                eyedists.append(ed)
                usable += ed >= 60

res = {"persons_detected": persons, "faces_detected": faces,
      "face_detection_rate": round(faces/max(1,persons), 4),
      "usable_for_recognition": usable,
      "usable_rate_of_persons": round(usable/max(1,persons), 4),
      "interocular_px_median": round(float(np.median(eyedists)), 1) if eyedists else 0,
      "interocular_px_p90": round(float(np.percentile(eyedists, 90)), 1) if eyedists else 0,
      "threshold_px": 60}
pathlib.Path("results/reid/face_feasibility.json").write_text(json.dumps(res, indent=2))
print(json.dumps(res, indent=2))
PY
python tools/face_feasibility.py

```

2.5 Gate and results table

Model	Rank-1	Rank-5	mAP	margin p05	ms/crop	Size
OSNet x0.25 fp32	fill	fill	fill	fill	fill	fill
OSNet x0.25 int8	fill	fill	fill	fill	fill	fill
Delta						

GATE 2

`results/reid/eval_reid-osnet-x025-int8.json` exists with Rank-1 above 0.80 and under 15 ms per crop at batch 32 on two threads. INT8 mAP is within 3 points of fp32. `face_feasibility.json` exists and gives you a measured, citable reason for the architecture choice in Q3.1a.

Phase 3. Edge Embedding Integration

Budget 2.5 h | **Branch** `phase/03-edge-embed` | **Gate** sightings reaching Postgres with 512-d vectors, at under 1.5 KB per sighting on the wire

3.1 Wire the real model in

`edge/embed.py` already looks for `VI_REID_MODEL` and falls back to the HSV placeholder. Point it at the exported model and confirm the fallback is no longer being used.

```
cp data/models/registry/reid-osnet-x025-int8.onnx data/models/reid-osnet-int8.onnx
echo 'export VI_REID_MODEL="$DATA/models/reid-osnet-int8.onnx"' >> .env
source .env

python - <<'PY'
import os; os.environ.setdefault("VI_ROOT", os.getcwd())
from common import telem; telem.init("data/logs/embed_smoke.jsonl")
from edge import embed
e = embed.Embedder()
assert e.kind == "onnx", f"still on fallback: {e.kind}"
print("kind:", e.kind, "dim:", e.dim)
PY
```

IF IT SAYS "FALLBACK"

The path is wrong or ONNX Runtime failed to load the model. Check `data/logs/embed_smoke.jsonl` for an `embedder_init_error` record. Do not proceed on the fallback: every downstream identity number will be meaningless, which is precisely the Round 1 failure mode repeating in a new stage.

3.2 Verify the embedding contract end to end

Three properties must hold before wiring into the cascade. Test them explicitly rather than assuming.

```
cat > tools/test_embed.py <<'PY'
#!/usr/bin/env python3
"""Contract tests for the edge embedder."""
import os, cv2, numpy as np, itertools
os.environ.setdefault("VI_ROOT", os.getcwd())
from common import telem; telem.init("data/logs/embed_test.jsonl")
from edge import detect, tracker, embed

d = detect.Detector(classes={1}); e = embed.Embedder()
t = tracker.Tracker("a_cam01", min_hits=3)
cap = cv2.VideoCapture("data/sim/a_cam01.mp4")
vecs, ids = [], []
for i in itertools.count():
    ok, f = cap.read()
    if not ok or len(vecs) > 40: break
    if i % 6: continue
    conf, _ = d(f, "a_cam01")
    act, _ = t.update(conf, f.shape)
    for tr in act:
        v = e(f, tr)
        if v is not None: vecs.append(v); ids.append(tr.id)

V = np.stack(vecs)
print("1. L2 norm == 1      :", np.allclose(np.linalg.norm(V, axis=1), 1.0, atol=1e-3))
print("2. dim == 512        :", V.shape[1] == 512)
print("3. dtype float32      :", V.dtype == np.float32)

same, diff = [], []
for i in range(len(V)):
    for j in range(i+1, len(V)):
        (same if ids[i]==ids[j] else diff).append(float(V[i] @ V[j]))
print(f"4. same-track sim mean {np.mean(same):.3f} (want high)")
print(f"5. cross-track sim mean {np.mean(diff):.3f} (want lower)")
print("6. separation          :", np.mean(same) - np.mean(diff) > 0.10)
PY
python tools/test_embed.py
```

Property 6 is the one that matters. If same-track similarity is not clearly above cross-track similarity on your own footage, the model is not transferring to your domain and no amount of threshold tuning downstream will fix it.

3.3 Confirm one embedding per tracklet, not per frame

The cascade is designed so the embedder runs only on quality-improving crops of confirmed tracks, and emits one descriptor when the tracklet closes. Verify the ratio, because it is a direct cost argument for Q4.3b.

```
edge/run.sh a & sleep 600; pkill -f "edge.agent"

python - <<'PY'
import json, collections
c = collections.Counter()
```

```

for line in open("data/logs/edge_a.jsonl"):
    d = json.loads(line); c[d.get("stage")] += 1
frames = c["detect_result"]
embeds = c["embed"]
tracks = c["track_end"]
print(f"frames inferred      : {frames}")
print(f"embed calls          : {embeds}")
print(f"tracklets closed       : {tracks}")
print(f"embeds per frame       : {embeds/max(1,frames):.3f}")
print(f"embeds per tracklet    : {embeds/max(1,tracks):.2f} (design target: <= 5)")
print(f"naive per-frame cost would be {frames/max(1,embeds):.1f}x higher")
PY

```

3.4 Measure the wire payload

This is the number that justifies edge-side embedding in Q3.1e and Q1.2.

```

python - <<'PY'
import json, statistics
sizes, dims = [], []
for line in open("data/logs/edge_a.jsonl"):
    d = json.loads(line)
    if d.get("stage") != "outbox": continue
for row in __import__("sqlite3").connect("data/edge_a.db").execute(
    "SELECT payload FROM outbox WHERE kind='sighting' LIMIT 500"):
    sizes.append(len(row[0]))
    dims.append(len(json.loads(row[0]).get("embedding", [])))
print(f"sightings sampled : {len(sizes)}")
print(f"payload bytes mean: {statistics.mean(sizes):.0f}")
print(f"payload bytes p95 : {sorted(sizes)[int(.95*len(sizes))]}")
print(f"embedding dim      : {set(dims)}")
print(f"vs a 640x360 JPEG crop at ~15000 bytes: {15000/statistics.mean(sizes):.1f}x smaller")
PY

```

Optional but worth doing: switch the wire format to float16 and re-measure. It halves the vector payload and, per the Phase 2 evaluation, costs almost nothing in Rank-1.

```

python - <<'PY'
import numpy as np
v = np.random.randn(512).astype(np.float32); v /= np.linalg.norm(v)
import json
f32 = len(json.dumps([round(float(x), 5) for x in v]))
f16 = len(np.asarray(v, np.float16).tobytes())
import base64
b64 = len(base64.b64encode(np.asarray(v, np.float16).tobytes()))
print(f"json float32 rounded : {f32} B")
print(f"raw float16          : {f16} B")
print(f"b64 float16           : {b64} B <- use this on the wire")
PY

```

3.5 Gate

GATE 3

All six contract tests pass. Embeds per tracklet at or below 5. `SELECT COUNT(*) FROM sightings` in Postgres is non-zero after a 10 minute run. Mean sighting payload under 1,500 bytes. `tools/verify.py --strict` still exits 0.

```

psql "$DATABASE_URL" -c \
"SELECT camera_id, COUNT(*), AVG(dwelling)::numeric(6,2), AVG(coherence)::numeric(5,3)
FROM sightings GROUP BY camera_id ORDER BY camera_id;"
python tools/verify.py data/logs/edge_a.jsonl --strict

```

Phase 4. Cloud Identity Plane and Threshold Calibration

Budget 3.5 h | **Branch** `phase/04-identity` | **Gate** a calibrated (tau, delta) pair chosen from a measured sweep, not guessed

This is the most important phase for PS-3. Everything else is plumbing; this is where the system decides who someone is, and where it learns to say "I do not know."

4.1 Enable pgvector and migrate the embedding column

```

psql "$DATABASE_URL" -c "CREATE EXTENSION IF NOT EXISTS vector;"
psql "$DATABASE_URL" <<'SQL'
ALTER TABLE sightings ADD COLUMN IF NOT EXISTS emb vector(512);
ALTER TABLE identities ADD COLUMN IF NOT EXISTS centroid_v vector(512);
UPDATE sightings SET emb = embedding::vector WHERE emb IS NULL AND embedding IS NOT NULL;
SQL
psql "$DATABASE_URL" -c "SELECT COUNT(*) FROM sightings WHERE emb IS NOT NULL;"

```

Do not build an index yet. Below roughly 100K vectors a sequential scan is faster than IVFFLAT, and Phase 5 will prove that with a curve rather than an opinion.

4.2 The resolver service

```

cat > cloud/resolver.py <<'PY'
#!/usr/bin/env python3
"""Consume unresolved sightings, assign identities, route ambiguity to review.
Runs as a loop beside the API. Deliberately separate from ingest: ingest must

```

```

stay fast and dumb, resolution can be slower and smarter."""
import os, time, json, uuid
import numpy as np, psycopg2
from psycopg2.extras import execute_values
from cloud import identity, adjudicate

DB = os.environ["DATABASE_URL"]
TAU = float(os.getenv("VI_TAU", 0.75))
TAULO = float(os.getenv("VI_TAU_LOW", 0.55))
DELTA = float(os.getenv("VI_DELTA", 0.10))

def load_gallery(cur):
    g = identity.Gallery(dim=512, tau=TAU, tau_low=TAULO, delta=DELTA)
    cur.execute("SELECT subject_id, centroid FROM identities WHERE centroid IS NOT NULL")
    for sid, c in cur.fetchall():
        g.upsert(sid, c)
    cur.execute("SELECT a_id, b_id FROM exclusions")
    for a, b in cur.fetchall():
        g.add_exclusion(a, b)
    cur.execute("SELECT subject_id FROM consent WHERE basis='revoked'")
    for (s,) in cur.fetchall():
        g.block_consent(s)
    return g

def covisible(cur, sighting_id):
    """Two tracklets overlapping in time on the SAME camera cannot be the same
    person. Free, very strong, and thrown away by most ReID systems."""
    cur.execute("""
        SELECT DISTINCT b.subject_id FROM sightings a JOIN sightings b
            ON a.camera_id = b.camera_id AND a.sighting_id <> b.sighting_id
            AND a.first_ts < b.last_ts AND b.first_ts < a.last_ts
            WHERE a.sighting_id = %s AND b.subject_id IS NOT NULL""", (sighting_id,))
    return {r[0] for r in cur.fetchall()}

def run_once(conn, limit=500):
    stats = {"match": 0, "new": 0, "review": 0}
    with conn, conn.cursor() as cur:
        g = load_gallery(cur)
        cur.execute("SELECT sighting_id, camera_id, site_id, embedding, coherence, "
                    "model_ver FROM sightings WHERE subject_id IS NULL "
                    "ORDER BY last_ts LIMIT %s", (limit,))
        for sid, cam, site, emb, coh, mv in cur.fetchall():
            r = g.resolve(np.asarray(emb, np.float32), coherence=coh or 1.0,
                          known_not=covisible(cur, sid))
            stats[r.decision] += 1
            if r.decision == identity.MATCH:
                cur.execute("UPDATE sightings SET subject_id=%s WHERE sighting_id=%s",
                            (r.subject_id, sid))
                cur.execute("UPDATE identities SET n_sightings=n_sightings+1, "
                            "last_seen=EXTRACT(EPOCH FROM now()) WHERE subject_id=%s",
                            (r.subject_id,))
            elif r.decision == identity.NEW:
                new_id = f"P{uuid.uuid4().hex[:12]}"
                cur.execute("INSERT INTO identities (subject_id, centroid, n_sightings, "
                            "first_seen, last_seen, embedding_ver) VALUES "
                            "(%s,%s,1,EXTRACT(EPOCH FROM now()),EXTRACT(EPOCH FROM now()),%s)",
                            (new_id, emb, mv))
                cur.execute("UPDATE sightings SET subject_id=%s WHERE sighting_id=%s",
                            (new_id, sid))
                g.upsert(new_id, emb)
            else:
                adjudicate.enqueue(cur, "identity_review", site or "unknown",
                                   dict(r.as_dict(), sighting_id=sid, camera_id=cam),
                                   priority=2, model_ver=mv)
                cur.execute("INSERT INTO audit_log (ts, actor, action, subject_id, site_id, detail) "
                            "VALUES (%s,'resolver','identity_resolve',%s,%s,%s)",
                            (time.time(), r.subject_id, site, json.dumps(r.as_dict())))

    return stats

if __name__ == "__main__":
    conn = psycopg2.connect(DB)
    while True:
        s = run_once(conn)
        print(json.dumps({"ts": time.time(), **s}), flush=True)
        time.sleep(5)

PY
chmod +x cloud/resolver.py
nohup python cloud/resolver.py > data/logs/resolver.jsonl 2>&1 &

```

4.3 Recompute centroids on a schedule (Q3.1d, drift)

An identity is not one vector. It is a running centroid over its recent sightings, which is what lets appearance change gradually without breaking the link.

```

cat > tools/recompute_centroids.py <<'PY'
#!/usr/bin/env python3
"""Recency-weighted centroid update. Old sightings decay, so an identity
tracks appearance change instead of being anchored to its first day."""
import os, time, numpy as np, psycopg2
HALF_LIFE_DAYS = float(os.getenv("VI_CENTROID_HALFLIFE", 30))
conn = psycopg2.connect(os.getenv("DATABASE_URL")); now = time.time()
with conn, conn.cursor() as cur:
    cur.execute("SELECT subject_id FROM identities WHERE needs_recompute "
                "OR updated_at < now() - interval '1 day'")
    for (sid,) in cur.fetchall():
        cur.execute("SELECT embedding, last_ts FROM sightings WHERE subject_id=%s "
                    "ORDER BY last_ts DESC LIMIT 50", (sid,))
        rows = cur.fetchall()
        if not rows: continue

```

```

V = np.array([r[0] for r in rows], np.float32)
age_d = np.array([(now - r[1]) / 86400 for r in rows], np.float32)
w = 0.5 ** (age_d / HALF_LIFE_DAYS)
c = (V * w[:, None]).sum(0) / w.sum()
c /= np.linalg.norm(c) + 1e-12
cur.execute("UPDATE identities SET centroid=%s, needs_recompute=false, "
            "n_sightings=%s, updated_at=now() WHERE subject_id=%s",
            (c.tolist(), len(rows), sid))
print("centroids recomputed")
PY
python tools/recompute_centroids.py

```

4.4 The calibration sweep: this is the data-driven core

Do not pick `tau` and `delta` by intuition. Sweep them against Market-1501 with ground truth, produce the surface, and choose the operating point that satisfies a stated precision target.

```

cat > tools/calibrate_thresholds.py <<'PY'
#!/usr/bin/env python3
"""Sweep (tau, delta) and report precision, recall, and review-queue load.
The output IS the answer to 'how did you choose your threshold'."""
import glob, json, pathlib, itertools
import numpy as np, cv2, onnxruntime as ort

MEAN=np.array([0.485,0.456,0.406],np.float32); STD=np.array([0.229,0.224,0.225],np.float32)
ROOT = pathlib.Path("data/datasets/Market-1501-v15.09.15")
def meta(p): n=pathlib.Path(p).name.split("_"); return int(n[0]), int(n[1][1])

so=ort.SessionOptions(); so.intra_op_num_threads=2
s=ort.InferenceSession("data/models/registry/reid-osnet-x025-int8.onnx",so,
                      providers=["CPUExecutionProvider"]); inp=s.get_inputs()[0].name
def emb(files, bs=32):
    o=[]
    for i in range(0,len(files),bs):
        b=[]
        for f in files[i:i+bs]:
            im=cv2.resize(cv2.imread(f),(128,256))[:,::-1].astype(np.float32)/255.
            b.append(((im-MEAN)/STD).transpose(2,0,1))
        v=s.run(None,{inp:np.ascontiguousarray(np.stack(b),dtype=np.float32)})[0]
        o.append(v/(np.linalg.norm(v,axis=1,keepdims=True)+1e-12))
    return np.vstack(o)

q=sorted(glob.glob(str(ROOT/"query"/"*.*.jpg"))[:1200])
g=sorted(f for f in glob.glob(str(ROOT/"bounding_box_test"/"*.*.jpg"))
        if not pathlib.Path(f).name.startswith("-1"))[:8000]
Q,G=emb(q),emb(g); qm=np.array([meta(f) for f in q]); gm=np.array([meta(f) for f in g])

# Precompute top-2 per query once; the sweep is then free.
tops=[]
for i in range(len(q)):
    sims=G@Q[i]
    valid=~((qm[:,0]==qm[i,0])&(gm[:,1]==gm[i,1]))
    sv,gp=sims[valid],gm[valid,0]; o=np.argsort(-sv)[:2]
    tops.append((float(sv[o[0]]), float(sv[o[1]]), int(gp[o[0]]==int(qm[i,0]))))

rows=[]
for tau,delta in itertools.product(np.arange(0.50,0.90,0.025),
                                   np.arange(0.00,0.20,0.01)):
    tp=fp=rev=new=0
    for t1,t2,correct in tops:
        if t1>=tau and (t1-t2)>=delta:
            tp+=correct; fp+= not correct
        elif t1<0.55: new+=1
        else: rev+=1
    dec=tp+fp
    rows.append({"tau":round(float(tau),3),"delta":round(float(delta),3),
                "precision":round(tp/dec,4) if dec else None,
                "recall":round(tp/len(tops),4),
                "auto_decide_rate":round(dec/len(tops),4),
                "review_rate":round(rev/len(tops),4),
                "new_rate":round(new/len(tops),4),
                "fp_count":fp})
pathlib.Path("results/identity").mkdir(parents=True,exist_ok=True)
import csv
with open("results/identity/threshold_sweep.csv","w",newline="") as f:
    w=csv.DictWriter(f,fieldnames=list(rows[0])); w.writeheader(); w.writerows(rows)

TARGET=0.99
ok=[r for r in rows if r["precision"] and r["precision"]>=TARGET]
best=max(ok,key=lambda r:r["recall"]) if ok else None
print(f"operating points meeting precision>={TARGET}: {len(ok)}")
print("chosen:", json.dumps(best,indent=2))
pathlib.Path("results/identity/chosen_operating_point.json").write_text(json.dumps(best,indent=2))
PY
python tools/calibrate_thresholds.py

```

Then apply the chosen point and record it as a versioned policy, not as an environment variable somebody will forget.

```

python - <<'PY'
import json, os, psycpg2
from cloud import adjudicate
p = json.load(open("results/identity/chosen_operating_point.json"))
conn = psycpg2.connect(os.environ["DATABASE_URL"])
with conn, conn.cursor() as cur:
    did, ver = adjudicate.emit_directive(cur, "*", "threshold",

```



```

{"scope": "identity", "tau": p["tau"], "delta": p["delta"],
 "source": "results/identity/threshold_sweep.csv"})
print("directive", did, "version", ver, "->", p)
PY

```

4.5 Cross-camera evaluation on your own topology

Market-1501 tells you the model is good. Your topology tells you the *system* is good, which is a different claim.

```

edge/run.sh a & sleep 900; pkill -f "edge.agent"
python tools/build_gt.py data/logs/edge_a.jsonl

cat > tools/eval_crosscam.py <<'PY'
#!/usr/bin/env python3
"""Did the resolver link the tracklets that ground truth says are the same
person, and did it avoid linking the ones it says are not?"""
import json, os, psycpg2, collections
gt = json.load(open("results/identity/ground_truth.json"))
conn = psycpg2.connect(os.environ["DATABASE_URL"])
cur = conn.cursor()
cur.execute("SELECT camera_id, track_id, subject_id FROM sightings WHERE subject_id IS NOT NULL")
assigned = {(c, t): s for c, t, s in cur.fetchall()}

tp = fn = 0; missing = 0
for p in gt["pairs"]:
    a = assigned.get((p["cam_a"], p["track_a"])); b = assigned.get((p["cam_b"], p["track_b"]))
    if a is None or b is None: missing += 1; continue
    tp += a == b; fn += a != b

groups = collections.defaultdict(set)
for (cam, tid), sid in assigned.items(): groups[sid].add((cam, tid))
gtmap = {}
for p in gt["pairs"]:
    gtmap[(p["cam_a"], p["track_a"])] = p["gid"]; gtmap[(p["cam_b"], p["track_b"])] = p["gid"]
fp = sum(1 for sid, members in groups.items()
        if len({gtmap[m] for m in members if m in gtmap}) > 1)

res = {"gt_pairs": len(gt["pairs"]), "unresolved_tracklets": missing,
      "correctly_linked": tp, "missed_links": fn, "over_merged_identities": fp,
      "link_recall": round(tp/max(1, tp+fn), 4),
      "identities_created": len(groups)}
open("results/identity/crosscam_eval.json", "w").write(json.dumps(res, indent=2))
print(json.dumps(res, indent=2))
PY
python tools/eval_crosscam.py

```

4.6 Domain shift measurement (Q3.1d)

Cameras 3 and 4 were built with deliberate illumination and resolution shift. Quantify the cost.

```

psql "$DATABASE_URL" <<'SQL' | tee results/identity/domain_shift.txt
SELECT s.camera_id,
       COUNT(*)           AS sightings,
       COUNT(s.subject_id) AS resolved,
       ROUND(100.0*COUNT(s.subject_id)/COUNT(*), 1) AS resolve_pct,
       ROUND(AVG(s.coherence):numeric, 3) AS mean_coherence
FROM sightings s GROUP BY s.camera_id ORDER BY s.camera_id;
SQL

```

GATE 4

`results/identity/threshold_sweep.csv` has at least 300 rows. `chosen_operating_point.json` meets the stated precision target. `crosscam_eval.json` shows link recall above 0.60 on your topology. The resolver log shows a non-zero `review` count, which proves the abstain path is live rather than decorative.

Phase 5. Gallery Scale Benchmark, 1K to 1M

Budget 2.5 h | **Branch** `phase/05-scale` | **Gate** a latency-versus-gallery-size curve with the flat-to-index crossover located empirically

Q3.1b and Q3.1c ask what changes between 1,000 and 1,000,000 identities. Most answers say "use a vector database" without saying when. This phase locates the crossover point with a measurement, which turns an opinion into an engineering decision.

5.1 Synthetic gallery generator

Real identities are scarce; the scaling behaviour is not identity-specific. Use synthetic vectors with realistic geometry: unit-norm, and clustered rather than uniformly random, because uniform random vectors in 512 dimensions are all nearly orthogonal and make any index look artificially good.

```

cat > tools/bench/make_gallery.py <<'PY'
#!/usr/bin/env python3
"""Synthetic gallery with realistic clustering. Each identity is a centroid
plus intra-class jitter, which is what a real ReID embedding space looks like."""
import sys, numpy as np, pathlib
n = int(sys.argv[1]); dim = 512
rng = np.random.default_rng(42)
n_clusters = max(8, n // 200)
centers = rng.normal(size=(n_clusters, dim)).astype(np.float32)
centers /= np.linalg.norm(centers, axis=1, keepdims=True)
assign = rng.integers(0, n_clusters, n)

```

```
V = centers[assign] + rng.normal(scale=0.35, size=(n, dim)).astype(np.float32)
V /= np.linalg.norm(V, axis=1, keepdims=True)
pathlib.Path("data/bench").mkdir(parents=True, exist_ok=True)
np.save(f"data/bench/gallery_{n}.npy", V.astype(np.float32))
print(f"{n} vectors, {V.nbytes/1e6:.1f} MB fp32, {V.nbytes/2e6:.1f} MB fp16")
PY
for n in 1000 10000 100000 1000000; do python tools/bench/make_gallery.py $n; done
```

5.2 Flat versus HNSW versus IVFFLAT

```
pip install hnswlib --break-system-packages

cat > tools/bench/scale_bench.py <<'PY'
#!/usr/bin/env python3
"""Query latency, recall@1 against exact search, memory, and build time,
across gallery sizes and index types."""
import time, json, csv, gc, pathlib, os
import numpy as np, hnswlib, psutil

proc = psutil.Process()
def rss_mb(): gc.collect(); return proc.memory_info().rss / 1e6

def pct(v, q): v = sorted(v); return v[(min(len(v)-1, int(q*len(v)))

rows = []
for n in (1000, 10000, 100000, 1000000):
    V = np.load(f"data/bench/gallery_{n}.npy")
    rng = np.random.default_rng(7)
    qi = rng.integers(0, n, 200)
    Q = V[qi] + rng.normal(scale=0.15, size=(200, V.shape[1])).astype(np.float32)
    Q /= np.linalg.norm(Q, axis=1, keepdims=True)

    # ---- exact flat (numpy matmul) -----
    base = rss_mb(); lat = []; truth = []
    for q in Q:
        t0 = time.perf_counter(); s = V @ q; top = int(np.argmax(s))
        lat.append((time.perf_counter()-t0)*1000); truth.append(top)
    rows.append({"n": n, "index": "flat_fp32", "build_s": 0.0,
                "p50_ms": round(pct(lat,.5),3), "p95_ms": round(pct(lat,.95),3),
                "recall@1": 1.0, "mem_mb": round(V.nbytes/1e6,1)})

    # ---- flat fp16 (half memory, tiny accuracy cost) -----
    Vh = V.astype(np.float16); lat = []; hit = 0
    for k, q in enumerate(Q):
        t0 = time.perf_counter()
        s = Vh @ q.astype(np.float16); top = int(np.argmax(s))
        lat.append((time.perf_counter()-t0)*1000); hit += top == truth[k]
    rows.append({"n": n, "index": "flat_fp16", "build_s": 0.0,
                "p50_ms": round(pct(lat,.5),3), "p95_ms": round(pct(lat,.95),3),
                "recall@1": round(hit/len(Q),4), "mem_mb": round(Vh.nbytes/1e6,1)})

    # ---- HNSW -----
    for M, ef in ((16, 64), (32, 128)):
        idx = hnswlib.Index(space="cosine", dim=V.shape[1])
        t0 = time.perf_counter()
        idx.init_index(max_elements=n, ef_construction=200, M=M)
        idx.add_items(V, np.arange(n)); build = time.perf_counter()-t0
        idx.set_ef(ef)
        lat = []; hit = 0
        for k, q in enumerate(Q):
            t0 = time.perf_counter(); lbl, _ = idx.knn_query(q, k=1)
            lat.append((time.perf_counter()-t0)*1000); hit += int(lbl[0][0]) == truth[k]
        rows.append({"n": n, "index": f"hnsw_M{M}_ef{ef}", "build_s": round(build,1),
                    "p50_ms": round(pct(lat,.5),3), "p95_ms": round(pct(lat,.95),3),
                    "recall@1": round(hit/len(Q),4),
                    "mem_mb": round(V.nbytes/1e6 + n*M*8/1e6, 1)})
        del idx
    del V, Vh; gc.collect()
    print(f"done n={n}")

pathlib.Path("results/scale").mkdir(parents=True, exist_ok=True)
with open("results/scale/scaling_curve.csv", "w", newline="") as f:
    w = csv.DictWriter(f, fieldnames=list(rows[0])); w.writeheader(); w.writerows(rows)
for r in rows: print(json.dumps(r))
PY
python tools/bench/scale_bench.py
```

5.3 The same question inside Postgres

Library benchmarks are useful; the number your architecture actually pays is the one through the database, including network and planner overhead.

```
psql "$DATABASE_URL" <<'SQL'
CREATE TABLE IF NOT EXISTS bench_vectors (id bigserial PRIMARY KEY, v vector(512));
TRUNCATE bench_vectors;
SQL

python - <<'PY'
import os, numpy as np, psycopg2
from psycopg2.extras import execute_values
V = np.load("data/bench/gallery_100000.npy")
conn = psycopg2.connect(os.environ["DATABASE_URL"])
with conn, conn.cursor() as cur:
    for i in range(0, len(V), 5000):
        execute_values(cur, "INSERT INTO bench_vectors (v) VALUES %s",
                        [(V[j].tolist(),) for j in range(i, min(i+5000, len(V))])
        print("inserted", min(i+5000, len(V)), flush=True)
PY
```

```
# sequential scan baseline
psql "$DATABASE_URL" -c "SET enable_indexscan=off;" \
-c "EXPLAIN ANALYZE SELECT id FROM bench_vectors
ORDER BY v <=> (SELECT v FROM bench_vectors LIMIT 1) LIMIT 5;"

# with IVFFLAT
psql "$DATABASE_URL" -c \
"CREATE INDEX ON bench_vectors USING ivfflat (v vector_cosine_ops) WITH (lists=316);"
psql "$DATABASE_URL" -c "SET ivfflat.probes = 10;" \
-c "EXPLAIN ANALYZE SELECT id FROM bench_vectors
ORDER BY v <=> (SELECT v FROM bench_vectors LIMIT 1) LIMIT 5;"

# with HNSW
psql "$DATABASE_URL" -c \
"CREATE INDEX ON bench_vectors USING hnsw (v vector_cosine_ops) WITH (m=16, ef_construction=64);"
psql "$DATABASE_URL" -c "EXPLAIN ANALYZE SELECT id FROM bench_vectors
ORDER BY v <=> (SELECT v FROM bench_vectors LIMIT 1) LIMIT 5;" \
| tee results/scale/pgvector_explain.txt
```

lists for IVFFLAT should be roughly $\sqrt{\text{rows}}$; 316 for 100K. Under-setting it degrades recall, over-setting it degrades build time and memory.

5.4 Storage cost model (Q3.1e)

```
cat > tools/bench/storage_model.py <<'PY'
#!/usr/bin/env python3
"""Per-identity storage decomposition, driven by measured row sizes."""
import csv, json, pathlib
DIM = 512
COMP = {
    "centroid_fp16": DIM * 2,
    "exemplars_5_fp16": DIM * 2 * 5,
    "sighting_rows_200": 200 * 120,
    "hnsw_graph_M16": 16 * 2 * 4 + 32,
    "audit_rows_50": 50 * 96,
}
per_id = sum(COMP.values())
rows = []
for n in (1_000, 10_000, 100_000, 1_000_000, 10_000_000):
    gb = per_id * n / 1e9
    rows.append({"identities": n,
                "bytes_per_identity": per_id,
                "total_gb": round(gb, 2),
                "storage_usd_month_at_0.10": round(gb * 0.10, 2),
                "ram_gb_if_index_resident": round(n * (DIM*2 + 16*2*4) / 1e9, 2),
                "ram_usd_month_at_5": round(n * (DIM*2 + 16*2*4) / 1e9 * 5, 2)})
pathlib.Path("results/scale").mkdir(parents=True, exist_ok=True)
with open("results/scale/storage_model.csv", "w", newline="") as f:
    w = csv.DictWriter(f, fieldnames=list(rows[0]))
    w.writeheader()
    w.writerows(rows)
print(json.dumps({"per_identity_bytes": per_id, "breakdown": COMP}, indent=2))
for r in rows: print(r)
PY
python tools/bench/storage_model.py
```

THE POINT TO MAKE FROM THIS TABLE

Storage is roughly 32 KB per identity, so 32 GB at one million, which is about three dollars a month. Storage is not the constraint. Resident RAM for the ANN index and the compute to re-embed every stored vector when the model changes are the constraints, and they scale differently. A model upgrade at one million identities is a one-million-crop re-inference job, and that is the cost nobody budgets for.

5.5 Results table and gate

Gallery	Index	p50	p95	recall@1	Memory
1K	flat fp32	fill	fill	1.000	fill
10K	flat fp32	fill	fill	1.000	fill
100K	flat fp32	fill	fill	1.000	fill
100K	hnsw M16	fill	fill	fill	fill
1M	flat fp32	fill	fill	1.000	fill
1M	hnsw M16	fill	fill	fill	fill

GATE 5

scaling_curve.csv covers all four gallery sizes and at least four index types. You can state, with a number from your own run, the gallery size at which flat search exceeds a 50 ms p95 budget. **storage_model.csv** exists. **pgvector_explain.txt** shows the planner switching from sequential scan to index scan.

Phase 6. Privacy, Consent, Retention, Audit

Budget 2.5 h | **Branch** `phase/06-privacy` | **Gate** a consent revocation that provably deletes edge pixels, cloud vectors, and derived centroids, with a receipt

6.1 Seed the consent model

```
psql "$DATABASE_URL" <<'SQL'
INSERT INTO consent (subject_id, basis, granted_at, retain_until)
SELECT subject_id, 'legitimate interest', now(),
       EXTRACT(EPOCH FROM now()) + 30*86400
FROM identities ON CONFLICT (subject_id) DO NOTHING;

SELECT basis, COUNT(*) FROM consent GROUP BY basis;
SQL
```

Three bases, and the distinction matters legally: `consent` is explicit and revocable at will, `legitimate interest` requires a balancing test and supports objection rather than revocation, `revoked` blocks all processing. The schema carries the basis on the record so that every query can be justified after the fact.

6.2 The DSAR endpoint (Q3.2b)

A data subject access request must return everything held about a person, and the act of answering it must itself be audited.

```
cat >> cloud/api.py <<'PY'

# -----
# DATA SUBJECT RIGHTS. Every one of these logs itself, because under GDPR
# Art.15 and DPDP s.11 the ACCESS is the sensitive operation, not just the
# storage. Looking someone up is the surveillance act.
# -----

@app.get("/privacy/subject/{subject_id}")
async def dsar_export(subject_id: str, authorization: str = Header(None),
                      x_actor: str = Header("unknown")):
    _auth(authorization)

    def _q(cur):
        cur.execute("INSERT INTO audit_log (ts, actor, action, subject_id, detail) "
                    "VALUES (%s,%s,'dsar_export',%s,%s)",
                    (time.time(), x_actor, subject_id, Json({"lawful_basis": "art15"})))
        cur.execute("SELECT subject_id, n_sightings, first_seen, last_seen, "
                    "embedding_ver FROM identities WHERE subject_id=%s", (subject_id,))
        ident = cur.fetchone()
        cur.execute("SELECT sighting_id, camera_id, site_id, first_ts, last_ts, "
                    "dwell_s, retain_until, consent_basis FROM sightings "
                    "WHERE subject_id=%s ORDER BY last_ts DESC LIMIT 1000", (subject_id,))
        s = cur.fetchall()
        cur.execute("SELECT ts, actor, action FROM audit_log WHERE subject_id=%s "
                    "ORDER BY ts DESC LIMIT 500", (subject_id,))
        a = cur.fetchall()
        return {"identity": ident, "sightings": s, "access_history": a}

    data = await asyncio.to_thread(_tx, _q)
    if not data["identity"]:
        raise HTTPException(status_code=404, detail="no such subject")
    return {"subject_id": subject_id, "generated_at": time.time(), **data}

@app.post("/privacy/subject/{subject_id}/revoke")
async def dsar_revoke(subject_id: str, authorization: str = Header(None),
                      x_actor: str = Header("unknown")):
    """Erasure. Fans out to every edge holding pixels via the control plane,
    then deletes cloud state, then marks derived centroids for recomputation."""
    _auth(authorization)
    from cloud import adjudicate

    def _revoke(cur):
        cur.execute("SELECT DISTINCT site_id FROM sightings WHERE subject_id=%s", (subject_id,))
        sites = [r[0] for r in cur.fetchall() if r[0]]
        directives = [adjudicate.emit_directive(cur, s, "consent_revoke",
                                                {"subject_id": subject_id})[0] for s in sites]
        cur.execute("UPDATE consent SET basis='revoked', revoked_at=now(), "
                    "retain_until=0 WHERE subject_id=%s", (subject_id,))
        cur.execute("SELECT COUNT(*) FROM sightings WHERE subject_id=%s", (subject_id,))
        n = cur.fetchone()[0]
        cur.execute("DELETE FROM sightings WHERE subject_id=%s", (subject_id,))
        cur.execute("DELETE FROM identities WHERE subject_id=%s", (subject_id,))
        receipt = {"subject_id": subject_id, "sightings_deleted": n,
                  "edge_directives": directives, "sites_notified": sites,
                  "completed_at": time.time()}
        cur.execute("INSERT INTO audit_log (ts, actor, action, subject_id, detail) "
                    "VALUES (%s,%s,'erasure',%s,%s)",
                    (time.time(), x_actor, subject_id, Json(receipt)))
        return receipt

    return await asyncio.to_thread(_tx, _revoke)
PY

kill -f "uvicorn cloud.api"; sleep 1
nohup uvicorn cloud.api:app --host 0.0.0.0 --port 8000 > data/logs/api.log 2>&1 &
sleep 3
```

6.3 The erasure drill

Run it end to end and keep the receipt. This is the artefact for Q3.2c.

```
SUBJ=$(psql "$DATABASE_URL" -tAc \
"SELECT subject_id FROM identities ORDER BY n_sightings DESC LIMIT 1")
echo "target subject: $SUBJ"
```

```
# 1. before
curl -s -H "authorization: Bearer $VI_API_TOKEN" -H "x-actor: drill" \
"$CLOUD_API/privacy/subject/$SUBJ" | tee results/privacy/dsar_before.json | head -30

sqlite3 data/edge_a.db "SELECT COUNT(*) AS edge_sightings FROM sightings WHERE subject_id='$SUBJ';"

# 2. revoke
curl -s -X POST -H "authorization: Bearer $VI_API_TOKEN" -H "x-actor: drill" \
"$CLOUD_API/privacy/subject/$SUBJ/revoke" | tee results/privacy/deletion_receipt.json

# 3. wait for the edge control poll to land (default interval 15 s)
sleep 40

# 4. after: all three must be zero
psql "$DATABASE_URL" -c "SELECT COUNT(*) AS cloud_sightings FROM sightings WHERE subject_id='$SUBJ';"
psql "$DATABASE_URL" -c "SELECT COUNT(*) AS cloud_identity FROM identities WHERE subject_id='$SUBJ';"
sqlite3 data/edge_a.db "SELECT COUNT(*) AS edge_sightings FROM sightings WHERE subject_id='$SUBJ';"

# 5. the audit trail survives the deletion, by design
psql "$DATABASE_URL" -c \
"SELECT to_timestamp(ts), actor, action FROM audit_log WHERE subject_id='$SUBJ' ORDER BY ts;" \
| tee results/privacy/audit_sample.txt

# 6. edge-side receipt
sqlite3 data/edge_a.db \
"SELECT datetime(ts,'unixepoch'), action, detail FROM audit_log
WHERE action='consent_purge' ORDER BY ts DESC LIMIT 3;"
```

6.4 Retention TTL under time compression

Waiting thirty days is not an option. Compress the clock instead.

```
psql "$DATABASE_URL" -c \
"UPDATE sightings SET retain_until = EXTRACT(EPOCH FROM now()) - 1
WHERE sighting_id IN (SELECT sighting_id FROM sightings LIMIT 100);"

python - <<'PY'
import os, psycopg2
from cloud import identity
conn = psycopg2.connect(os.environ["DATABASE_URL"])
with conn, conn.cursor() as cur:
    n = identity.sweep_expired(cur)
    print(f"expired sweep affected {n} identities needing centroid recompute")
PY
python tools/recompute_centroids.py
psql "$DATABASE_URL" -c "SELECT COUNT(*) FROM identities WHERE needs_recompute;"
```

THE DERIVED-ARTEFACT PROBLEM, STATED HONESTLY

Deleting a sighting does not delete its influence. A centroid computed from ten sightings still encodes information about all ten, so erasure must trigger recomputation from the surviving set, not merely row removal. The `needs_recompute` flag exists for exactly this, and the fact that the count above is non-zero after the sweep is the evidence that the system understands the distinction. This is the strongest available answer to Q3.2d, because it names a problem most designs do not notice.

GATE 6

`deletion_receipt.json` names the sites notified and the directive ids. Cloud sightings, cloud identity, and edge sightings for the subject are all zero after the poll interval. `audit_sample.txt` still contains the subject's access history, proving deletion is auditable after the fact. `needs_recompute` is non-zero after the TTL sweep.

Phase 7. False Positive Remediation Drill

Budget 1.5 h | **Branch** `phase/07-fpdrill` | **Gate** a reproduced false match, an exclusion that lands at the edge in under 60 s, and a measured systemic fix

Q3.3 gives a scenario: a customer reports that John was matched as David five times last week. The question has three parts, and each maps to a different time horizon. Answering it well means demonstrating all three, not choosing one.

Horizon	Action	Cost of being wrong
Minutes	hard exclusion on the pair	none, it is a constraint not a model change
Days	raise <code>delta</code> for the affected gallery	some recall, scoped to one cohort
Weeks	hard-example mining and retrain	expensive, needs canary protection

7.1 Reproduce the failure deliberately

You cannot claim to have fixed something you never observed. Force the condition by finding the two identities in your own gallery with the smallest separation, which is exactly the John/David geometry.

```
cat > tools/find_confusable.py <<'PY'
#!/usr/bin/env python3
"""Find the identity pairs most at risk of being confused: high cosine
similarity between distinct centroids. These ARE your future support tickets."""
import os, json, numpy as np, psycopg2, pathlib
conn = psycopg2.connect(os.environ["DATABASE_URL"]); cur = conn.cursor()
cur.execute("SELECT subject_id, centroid FROM identities WHERE centroid IS NOT NULL")
rows = cur.fetchall()
ids = [r[0] for r in rows]
```

```

V = np.array([r[1] for r in rows], np.float32)
V /= np.linalg.norm(V, axis=1, keepdims=True) + 1e-12
S = V @ V.T
np.fill_diagonal(S, -1)
pairs = []
for i in range(len(ids)):
    j = int(np.argmax(S[i]))
    if j > i:
        pairs.append({"a": ids[i], "b": ids[j], "sim": round(float(S[i, j]), 4)})
pairs.sort(key=lambda p: -p["sim"])
out = {"n_identities": len(ids), "top_confusable": pairs[:20],
      "pairs_above_0.75": sum(1 for p in pairs if p["sim"] > 0.75),
      "pairs_above_0.85": sum(1 for p in pairs if p["sim"] > 0.85)}
pathlib.Path("results/privacy").mkdir(parents=True, exist_ok=True)
pathlib.Path("results/privacy/fp_diagnosis.json").write_text(json.dumps(out, indent=2))
print(json.dumps(out, indent=2))
PY
python tools/find_confusable.py

```

7.2 Diagnose it the way you would in a support ticket (Q3.3a)

The diagnosis is not "the model is bad." It is a specific, checkable chain: pull the five events, recompute the similarities, and look at the margin.

```

JOHN=$(python -c "import json;print(json.load(open('results/privacy/fp_diagnosis.json'))['top_confusable'][0]['a'])")
DAVID=$(python -c "import json;print(json.load(open('results/privacy/fp_diagnosis.json'))['top_confusable'][0]['b'])")
echo "confusable pair: $JOHN <-> $DAVID"

psql "$DATABASE_URL" <<SQL | tee results/privacy/fp_evidence.txt
SELECT s.sighting_id, s.camera_id, to_timestamp(s.last_ts) AS seen,
       s.coherence, s.dwell_s, s.model_ver
FROM sightings s WHERE s.subject_id IN ('$JOHN', '$DAVID')
ORDER BY s.last_ts DESC LIMIT 20;

SELECT a.subject_id, b.subject_id,
       ROUND((1 - (a.centroid_v <=> b.centroid_v))::numeric, 4) AS cosine_sim
FROM identities a, identities b
WHERE a.subject_id='$JOHN' AND b.subject_id='$DAVID';
SQL

```

The diagnostic questions in order, because the answer changes the fix:

1. **Is the margin small?** If top-1 minus top-2 is under **delta**, the resolver should have abstained and the operating point is mis-set.
2. **Is coherence low?** A tracklet whose own views disagreed produced a noisy descriptor; the fix is upstream, in crop quality gating.
3. **Are they co-visible?** If both appear in the same frame at the same time, they are provably different people and the constraint was not applied.
4. **Same camera, similar lighting?** Then it is domain-specific and a cohort-scoped threshold beats a global one.

7.3 The immediate fix, and measure how fast it lands (Q3.3b)

```

python - <<'PY'
import os, time, json, psycopg2, pathlib
from cloud import adjudicate
john = json.load(open("results/privacy/fp_diagnosis.json"))["top_confusable"][0]["a"]
david = json.load(open("results/privacy/fp_diagnosis.json"))["top_confusable"][0]["b"]
conn = psycopg2.connect(os.environ["DATABASE_URL"])
t0 = time.time()
with conn, conn.cursor() as cur:
    cur.execute("INSERT INTO exclusions (a_id,b_id,reason) VALUES (%s,%s,%s) "
               "ON CONFLICT DO NOTHING", (john, david, "customer_report:ticket-4471"))
    did, ver = adjudicate.emit_directive(cur, "*", "exclusion_pair",
                                         {"a_id": john, "b_id": david,
                                          "reason": "customer_report:ticket-4471"})
pathlib.Path("results/privacy/exclusion_emitted.json").write_text(
    json.dumps({"directive_id": did, "version": ver, "emitted_at": t0,
               "a": john, "b": david}, indent=2))
print("emitted", did, "at", t0)
PY

# measure edge propagation latency
python - <<'PY'
import json, sqlite3, time
meta = json.load(open("results/privacy/exclusion_emitted.json"))
deadline = time.time() + 120
while time.time() < deadline:
    r = sqlite3.connect("data/edge_a.db").execute(
        "SELECT applied_ts, ok FROM applied_directives WHERE directive_id=?",
        (meta["directive_id"],)).fetchone()
    if r:
        lat = r[0] - meta["emitted_at"]
        out = {"propagation_seconds": round(lat, 2), "applied_ok": bool(r[1]),
               "directive_id": meta["directive_id"]}
        open("results/privacy/exclusion_latency.json", "w").write(json.dumps(out, indent=2))
        print(json.dumps(out, indent=2)); break
    time.sleep(2)
else:
    print("TIMEOUT: directive never applied. Check the inbox thread.")
PY

```

WHY THIS BEATS RAISING THE GLOBAL THRESHOLD

Raising `tau` globally to stop one bad pair degrades recall for every other customer, and you will not notice because the metric that would show it is averaged across the fleet. The exclusion is surgical: it costs exactly the two identities involved and nothing else, and it is live within one control poll. Say that sentence in the Q3.3b answer.

7.4 The systemic change, measured (Q3.3c)

The systemic fix is not "retrain." It is to route the ambiguous band to review instead of committing to it, and then to prove the trade with a number.

```
python - <<'PY'
"""What does raising delta actually buy, and what does it cost?
Re-reads the Phase 4 sweep, so this is free."""
import csv, json, pathlib
rows = list(csv.DictReader(open("results/identity/threshold_sweep.csv")))
chosen = json.load(open("results/identity/chosen_operating_point.json"))
tau = chosen["tau"]
band = [r for r in rows if abs(float(r["tau"]) - tau) < 1e-6 and r["precision"]]
band.sort(key=lambda r: float(r["delta"]))
print(f"{'delta':>6} {'precision':>10} {'recall':>8} {'auto%':>7} {'review%':>8} {'FPs':>5}")
out = []
for r in band:
    print(f"{float(r['delta']):6.2f} {float(r['precision']):10.4f} "
          f"{float(r['recall']):8.4f} {float(r['auto_decide_rate'])*100:6.1f}% "
          f"{float(r['review_rate'])*100:7.1f}% {r['fp_count']:>5}")
    out.append(r)
base = out[0]
for r in out:
    if int(r["fp_count"]) <= int(base["fp_count"]) // 2:
        print(f"\nHalving false positives costs {(float(base['auto_decide_rate'])-float(r['auto_decide_rate']))*100:.1f} "
              f"points of automation, at delta={r['delta']}")
        break
with open("results/lifecycle/margin_impact.csv", "w", newline="") as f:
    w = csv.DictWriter(f, fieldnames=list(out[0])); w.writeheader(); w.writerows(out)
PY
```

GATE 7

`fp_diagnosis.json` lists confusable pairs from your real gallery. `exclusion_latency.json` shows propagation under 60 seconds with `applied_ok` true. `margin_impact.csv` lets you state, with a number, what halving false positives costs in automation rate.

Phase 8. PS-4 Detection Architecture Benchmark

Budget 3.0 h | **Branch** `phase/08-objects` | **Gate** a measured comparison of monolithic versus tiered detection on identical inputs

Q4.1 asks whether 200-plus object classes should be one model or many, and whether classification or embedding retrieval is the right mechanism. Both are answerable by benchmark rather than by argument.

8.1 Build the class inventory

```
python - <<'PY'
import yaml, pathlib, json
base = yaml.safe_load(open("data/datasets/objects/ppe/data.yaml"))
inv = {
    "tier_common": ["person", "helmet", "vest", "gloves", "boots", "goggles"],
    "tier_site": ["ladder", "scaffold", "cone", "barrier", "extinguisher", "toolbox"],
    "tier_custom": [], # populated per customer at runtime, Phase 9
    "source_classes": base.get("names", []),
}
pathlib.Path("edge/classes.yaml").write_text(yaml.safe_dump(inv, sort_keys=False))
print(json.dumps(inv, indent=2))
PY
```

The three-tier split is the architecture answer in miniature. Common classes are worth putting in the base model because every customer needs them. Site classes are worth a shared secondary model. Custom classes must not require a retrain, because a retrain has a lead time measured in weeks and Q4.1c gives you one day.

8.2 Strategy A versus Strategy B, benchmarked

Strategy A is one model that knows everything. Strategy B is a cheap class-agnostic proposal stage followed by a targeted classifier. Measure both on identical frames.

```
cat > tools/bench/arch_benchmark.py <<'PY'
#!/usr/bin/env python3
"""Strategy A (monolithic multi-class detector) vs
Strategy B (class-agnostic proposals -> lightweight classifier).
Same frames, same machine, same thread budget. The only variable is topology."""
import os, time, json, csv, glob, pathlib, statistics
import numpy as np, cv2, onnxruntime as ort
os.environ.setdefault("VI_ROOT", os.getcwd())
from edge import detect

FRAMES = sorted(glob.glob("data/datasets/objects/ppe/valid/images/*.jpg"))[:200]
assert FRAMES, "no validation frames found"
```



```

def pct(v, q): v = sorted(v); return v[min(len(v)-1, int(q*len(v)))]

# ----- Strategy A: one detector, all classes -----
det_all = detect.Detector(classes=None)
latA, nA = [], []
for f in FRAMES:
    img = cv2.imread(f)
    t0 = time.perf_counter(); c, u = det_all(img, "bench")
    latA.append((time.perf_counter()-t0)*1000); nA.append(len(c))

# ----- Strategy B: proposals then classify -----
# Stage 1 runs at reduced resolution and keeps only geometry.
det_prop = detect.Detector(size=256, conf_low=0.20, conf_high=0.25, classes=None)
so = ort.SessionOptions(); so.intra_op_num_threads = 1
so.graph_optimization_level = ort.GraphOptimizationLevel.ORT_ENABLE_ALL
clf = ort.InferenceSession("data/models/registry/reid-osnet-x025-int8.onnx", so,
                           providers=["CPUExecutionProvider"]) # stand-in classifier
cinp = clf.get_inputs()[0].name
MEAN=np.array([0.485,0.456,0.406],np.float32); STD=np.array([0.229,0.224,0.225],np.float32)

latB, nB, s1, s2 = [], [], [], []
for f in FRAMES:
    img = cv2.imread(f)
    t0 = time.perf_counter()
    c, u = det_prop(img, "bench")
    t1 = time.perf_counter()
    crops = []
    for d in (c + u)[:12]:
        x1,y1,x2,y2 = [int(v) for v in d.box]
        cr = img[max(0,y1):y2, max(0,x1):x2]
        if cr.size == 0: continue
        cr = cv2.resize(cr, (128,256))[:,::-1].astype(np.float32)/255.
        crops.append(((cr-MEAN)/STD).transpose(2,0,1))
    if crops:
        clf.run(None, {cinp: np.ascontiguousarray(np.stack(crops), dtype=np.float32)})
    t2 = time.perf_counter()
    latB.append((t2-t0)*1000); s1.append((t1-t0)*1000); s2.append((t2-t1)*1000)
    nB.append(len(crops))

rows = [
    {"strategy": "A_monolithic", "stage": "detect_384_allclass",
     "p50_ms": round(pct(latA, .5), 1), "p95_ms": round(pct(latA, .95), 1),
     "mean_ms": round(statistics.mean(latA), 1),
     "objects_per_frame": round(statistics.mean(nA), 2),
     "models_loaded": 1, "add_class_requires": "full retrain + redeploy"},
    {"strategy": "B_tiered", "stage": "propose_256 + classify_crops",
     "p50_ms": round(pct(latB, .5), 1), "p95_ms": round(pct(latB, .95), 1),
     "mean_ms": round(statistics.mean(latB), 1),
     "objects_per_frame": round(statistics.mean(nB), 2),
     "models_loaded": 2, "add_class_requires": "classifier head or gallery only"},
    {"strategy": "B_breakdown", "stage": "stage1_propose",
     "p50_ms": round(pct(s1, .5), 1), "p95_ms": round(pct(s1, .95), 1),
     "mean_ms": round(statistics.mean(s1), 1), "objects_per_frame": None,
     "models_loaded": None, "add_class_requires": None},
    {"strategy": "B_breakdown", "stage": "stage2_classify",
     "p50_ms": round(pct(s2, .5), 1), "p95_ms": round(pct(s2, .95), 1),
     "mean_ms": round(statistics.mean(s2), 1), "objects_per_frame": None,
     "models_loaded": None, "add_class_requires": None},
]
pathlib.Path("results/objects").mkdir(parents=True, exist_ok=True)
with open("results/objects/arch_benchmark.csv", "w", newline="") as fh:
    w = csv.DictWriter(fh, fieldnames=list(rows[0])); w.writeheader(); w.writerows(rows)
for r in rows: print(json.dumps(r))
PY
python tools/bench/arch_benchmark.py

```

8.3 The scaling argument, extrapolated from the measurement

```

python - <<'PY'
"""Extrapolate both strategies to 200+ classes using the measured per-stage
costs. Strategy A grows with class count in the head; Strategy B grows only in
the classifier, which sees far fewer pixels."""
import csv, json
r = {row["stage"]: row for row in csv.DictReader(open("results/objects/arch_benchmark.csv"))}
a = float(r["detect_384_allclass"]["mean_ms"])
s1 = float(r["stage1_propose"]["mean_ms"])
s2 = float(r["stage2_classify"]["mean_ms"])
print(f'{"classes":>8} {"StratA ms":>10} {"StratB ms":>10} {"winner":>8}')
out=[]
for k in (20, 50, 100, 200, 500):
    # A: detection head scales roughly linearly in class count beyond the backbone
    ma = a * (1 + 0.0018 * (k - 20))
    # B: proposal stage is class-agnostic and flat; classifier head is cheap per class
    mb = s1 + s2 * (1 + 0.0004 * (k - 20))
    out.append({"classes":k, "strategy_a_ms":round(ma,1), "strategy_b_ms":round(mb,1),
               "winner":"A" if ma<mb else "B"})
    print(f'{k:>8} {ma:>10.1f} {mb:>10.1f} {("A" if ma<mb else "B"):>8}')
json.dump(out, open("results/objects/arch_extrapolation.json", "w"), indent=2)
PY

```

THE HONEST FRAMING FOR Q4.1D

Strategy B is not universally better. It wins on class count and on time to add a class, and it loses on small or heavily occluded objects, because a proposal stage that misses an object denies the classifier any chance to recover it. Recall in stage 1 is a hard ceiling on the whole pipeline. State that ceiling as the deciding trade-off rather than presenting B as free.

GATE 8

`arch_benchmark.csv` contains four rows with real timings. `arch_extrapolation.json` identifies the class count at which the two strategies cross. You can quote stage-1 recall as the limiting factor.

Phase 9. New Customer Class in Under One Day

Budget 2.0 h | **Branch** `phase/09-newclass` | **Gate** forklift detection live at the edge, from zero, with a timestamped log and no retrain

Q4.1c is the question with a deadline attached. The answer is not "we retrain quickly." It is that **the class does not go into the model at all**. Open-set classes live in an embedding gallery, and adding one is a data write plus a control directive, both of which take seconds.

This is the same mechanism as the identity gallery from Phase 4, which is the connection worth naming: a customer-specific object class and a person identity are the same problem, so they get the same machinery.

9.1 Start the clock

```
date -Isseconds | tee results/objects/new_class_t0.txt
mkdir -p results/objects
```

9.2 Build the prototype gallery from a handful of examples

```
cat > tools/add_class.py <<'PY'
#!/usr/bin/env python3
"""Add an open-set class from N reference crops. No training, no redeploy.
Usage: python tools/add_class.py forklift data/datasets/objects/forklift/train/images 25"""
import sys, glob, json, time, pathlib
import numpy as np, cv2, onnxruntime as ort

name, src, k = sys.argv[1], sys.argv[2], int(sys.argv[3] if len(sys.argv) > 3 else 20)
MEAN=np.array([0.485,0.456,0.406],np.float32); STD=np.array([0.229,0.224,0.225],np.float32)

so = ort.SessionOptions(); so.intra_op_num_threads = 2
so.graph_optimization_level = ort.GraphOptimizationLevel.ORT_ENABLE_ALL
s = ort.InferenceSession("data/models/registry/reid-osnet-x025-int8.onnx", so,
                        providers=["CPUExecutionProvider"])
inp = s.get_inputs()[0].name

files = sorted(glob.glob(f"{src}/*.jpg"))[:k]
assert files, f"no images under {src}"
vecs = []
for f in files:
    img = cv2.imread(f)
    lbl = f.replace("/images/", "/labels/").rsplit(".", 1)[0] + ".txt"
    boxes = []
    if pathlib.Path(lbl).exists():
        h, w = img.shape[:2]
        for line in open(lbl):
            p = line.split()
            if len(p) < 5: continue
            cx, cy, bw, bh = [float(x) for x in p[1:5]]
            boxes.append([int((cx-bw/2)*w), int((cy-bh/2)*h),
                          int((cx+bw/2)*w), int((cy+bh/2)*h)])
    for x1, y1, x2, y2 in (boxes or [[0, 0, img.shape[1], img.shape[0]]]):
        cr = img[max(0,y1):y2, max(0,x1):x2]
        if cr.size == 0 or (x2-x1) < 20 or (y2-y1) < 20: continue
        cr = cv2.resize(cr, (128, 256))[:, :, :-1].astype(np.float32)/255.
        x = np.ascontiguousarray(((cr-MEAN)/STD).transpose(2,0,1)[None], dtype=np.float32)
        v = s.run(None, {inp: x})[0][0]
        vecs.append(v/(np.linalg.norm(v)+1e-12))

V = np.stack(vecs)
proto = V.mean(0); proto /= np.linalg.norm(proto)
# Intra-class spread sets the acceptance radius. A tight class gets a tight
# threshold; a visually diverse class gets a loose one. Do not hardcode 0.7.
sims = V @ proto
out = {"class": name, "n examples": len(V),
      "prototype": [round(float(x), 6) for x in proto],
      "exemplars": [[round(float(x), 6) for x in v] for v in V[:8]],
      "intra_sim_mean": round(float(sims.mean()), 4),
      "intra_sim_p05": round(float(np.percentile(sims, 5)), 4),
      "suggested_threshold": round(float(np.percentile(sims, 5)) - 0.05, 4),
      "created_at": time.time()}
pathlib.Path(f"results/objects/class_{name}.json").write_text(json.dumps(out, indent=2))
print(json.dumps({k2: v for k2, v in out.items()
                  if k2 not in ("prototype", "exemplars")}, indent=2))

PY
chmod +x tools/add_class.py
python tools/add_class.py forklift data/datasets/objects/forklift/train/images 25
```

9.3 Ship it to the edge through the control plane

```
python - <<'PY'
import os, json, time, psycpg2, pathlib
from cloud import adjudicate
c = json.load(open("results/objects/class_forklift.json"))
conn = psycpg2.connect(os.environ["DATABASE_URL"])
t0 = time.time()
```

```

with conn, conn.cursor() as cur:
    did, ver = adjudicate.emit_directive(cur, "*", "class_manifest", {
        "classes": [{ "name": "forklift", "mode": "open set",
            "threshold": c["suggested_threshold"]}],
        "reference_embeddings": { "forklift": c["prototype"] },
        "exemplars": { "forklift": c["exemplars"] },
    })
pathlib.Path("results/objects/new_class_directive.json").write_text(
    json.dumps({"directive_id": did, "version": ver, "emitted_at": t0}, indent=2))
print("emitted", did, "version", ver)
PY

sleep 30
sqlite3 data/edge_a.db \
    "SELECT directive_id, kind, version, datetime(applied_ts, 'unixepoch'), ok
    FROM applied_directives WHERE kind='class_manifest' ORDER BY applied_ts DESC LIMIT 3;"

```

9.4 Evaluate the new class without a single training step

```

cat > tools/eval_openset.py <<'PY'
#!/usr/bin/env python3
"""Precision and recall for an open-set class against a held-out split, and
against negatives drawn from a different class. Answers Q4.1b with numbers."""
import sys, glob, json, pathlib
import numpy as np, cv2, onnxruntime as ort
MEAN=np.array([0.485,0.456,0.406],np.float32); STD=np.array([0.229,0.224,0.225],np.float32)
name = sys.argv[1]
meta = json.load(open(f"results/objects/class_{name}.json"))
proto = np.array(meta["prototype"], np.float32); thr = meta["suggested_threshold"]

so=ort.SessionOptions(); so.intra_op_num_threads=2
s=ort.InferenceSession("data/models/registry/reid-osnet-x025-int8.onnx",so,
    providers=["CPUExecutionProvider"]); inp=s.get_inputs()[0].name
def emb(f):
    im=cv2.imread(f)
    if im is None: return None
    im=cv2.resize(im,(128,256))[:,:,:-1].astype(np.float32)/255.
    v=s.run(None,{inp:np.ascontiguousarray((im-MEAN)/STD).transpose(2,0,1)[None],dtype=np.float32)})[0][0]
    return v/(np.linalg.norm(v)+1e-12)

pos = sorted(glob.glob(f"data/datasets/objects/{name}/valid/images/*.jpg"))[:120]
neg = sorted(glob.glob("data/datasets/objects/ppe/valid/images/*.jpg"))[:120]
tp = sum(1 for f in pos if (v:=emb(f)) is not None and v @ proto >= thr)
fp = sum(1 for f in neg if (v:=emb(f)) is not None and v @ proto >= thr)
res = {"class": name, "threshold": thr, "n_pos": len(pos), "n_neg": len(neg),
    "true_positives": tp, "false_positives": fp,
    "recall": round(tp/max(1,len(pos)), 4),
    "precision": round(tp/max(1, tp+fp), 4),
    "training_steps": 0, "gpu hours": 0.0}
pathlib.Path("results/objects/openset_eval.json").write_text(json.dumps(res, indent=2))
print(json.dumps(res, indent=2))
PY
python tools/eval_openset.py forklift

```

9.5 Stop the clock and write the timeline

```

python - <<'PY'
import json, datetime, pathlib
t0 = open("results/objects/new_class_t0.txt").read().strip()
d = json.load(open("results/objects/new_class_directive.json"))
e = json.load(open("results/objects/openset_eval.json"))
t0dt = datetime.datetime.fromisoformat(t0)
elapsed = (datetime.datetime.fromtimestamp(d["emitted_at"], t0dt.tzinfo) - t0dt).total_seconds()
md = f"""# New class: forklift

| Step | Detail |
|---|---|
| Started | {t0} |
| Reference examples collected | {json.load(open('results/objects/class_forklift.json'))['n_examples']} |
| Training steps | 0 |
| GPU hours | 0 |
| Directive emitted | version {d['version']} |
| Wall clock to live | {elapsed/60:.1f} minutes |
| Recall on held-out split | {e['recall']} |
| Precision against negatives | {e['precision']} |
| Acceptance threshold | {e['threshold']} (derived from intra-class p05, not hardcoded) |

The class never entered the detector. Stage 1 proposes class-agnostic regions,
stage 2 embeds each crop, and membership is a cosine comparison against a
prototype held in a gallery. Adding a class is therefore a database write plus a
control-plane directive, which is why the deadline is minutes rather than weeks.

The cost is a lower ceiling than a trained head would give. A fine-tuned
detector would beat this on precision. The trade is deliberate: open-set gets a
customer to working on day one, and the class can be promoted into the base
model on the next scheduled retrain if usage justifies it.
"""
pathlib.Path("results/objects/new_class_timeline.md").write_text(md)
print(md)
PY

```

GATE 9

`new_class_timeline.md` shows wall clock under 60 minutes with zero training steps. `openset_eval.json` reports recall above 0.60 and precision above 0.70. The edge `applied_directives` table contains the `class_manifest` row with `ok=1`.

Phase 10. Model Lifecycle: Canary, Cohorts, Per-Tenant Rollback

Budget 3.0 h | **Branch** `phase/10-lifecycle` | **Gate** Customer A rolled back while Customer C stays on the new model, both verified from the edge databases

Q4.2 describes a deploy where three customers get three different outcomes: A gets more false positives, B gets slower, C is happy. The reason it shipped is that a single global accuracy number averaged A's regression against C's improvement. The fix is not a better test set. It is cohort-scoped metrics and per-tenant version pinning.

10.1 Model registry

```
cat > tools/register_model.py <<'PY'
#!/usr/bin/env python3
"""Register a model version with a checksum. Nothing ships unregistered,
because a rollback needs a known-good artefact to roll back TO."""
import sys, hashlib, os, json, psycopg2, pathlib
path, ver = sys.argv[1], sys.argv[2]
sha = hashlib.sha256(pathlib.Path(path).read_bytes()).hexdigest()
size = os.path.getsize(path)
conn = psycopg2.connect(os.environ["DATABASE_URL"])
with conn, conn.cursor() as cur:
    cur.execute("""CREATE TABLE IF NOT EXISTS model_registry (
        model_ver text PRIMARY KEY, path text, sha256 text, bytes bigint,
        registered_at timestampz DEFAULT now(), notes text)""")
    cur.execute("INSERT INTO model_registry (model_ver, path, sha256, bytes) "
        "VALUES (%s,%s,%s,%s) ON CONFLICT (model_ver) DO NOTHING",
        (ver, path, sha, size))
print(json.dumps({"model_ver": ver, "sha256": sha[:16], "mb": round(size/1e6, 1)}))
PY
chmod +x tools/register_model.py

python tools/register_model.py data/models/rf-detr-nano-int8.onnx v1.0-baseline
```

10.2 Manufacture a regression, on purpose

To demonstrate a rollback you need something worth rolling back from. Create a v1.1 that is genuinely worse for one cohort by lowering its confidence floor, which raises false positives exactly the way the scenario describes.

```
cp data/models/rf-detr-nano-int8.onnx data/models/registry/rf-detr-nano-v1.1.onnx
python tools/register_model.py data/models/registry/rf-detr-nano-v1.1.onnx v1.1-candidate

python - <<'PY'
import os, psycopg2
from cloud import adjudicate
conn = psycopg2.connect(os.environ["DATABASE_URL"])
with conn, conn.cursor() as cur:
    # Canary: cohort A only. The whole point is that C is untouched.
    did, ver = adjudicate.emit_directive(cur, "a", "model_pin", {
        "model_ver": "v1.1-candidate",
        "model_uri": "file://data/models/registry/rf-detr-nano-v1.1.onnx",
        "sha256": "recompute-on-edge",
        "conf_high": 0.30}) # deliberately loosened -> more false positives
    print("canary to site a:", did, ver)
PY
```

10.3 Feed the cohort metrics table

```
cat > tools/simulate_verdicts.py <<'PY'
#!/usr/bin/env python3
"""Populate cohort metrics from the adjudication queue. In production these
come from real reviewer verdicts; here we synthesise a realistic mix so the
canary gate has something to evaluate."""
import os, random, psycopg2, sys
random.seed(11)
cohort, ver, n, fp_rate = sys.argv[1], sys.argv[2], int(sys.argv[3]), float(sys.argv[4])
conn = psycopg2.connect(os.environ["DATABASE_URL"])
with conn, conn.cursor() as cur:
    fp = sum(1 for _ in range(n) if random.random() < fp_rate)
    cur.execute("""INSERT INTO cohort_metrics (cohort, model_ver, day, tp, fp)
        VALUES (%s,%s,CURRENT_DATE,%s,%s)
        ON CONFLICT (cohort, model_ver, day) DO UPDATE SET
            tp = cohort_metrics.tp + EXCLUDED.tp, fp = cohort_metrics.fp + EXCLUDED.fp""",
        (cohort, ver, n - fp, fp))
    cur.execute("""INSERT INTO cohort_latency (cohort, model_ver, day, p50_latency_ms, p95_latency_ms)
        VALUES (%s,%s,CURRENT_DATE,%s,%s)
        ON CONFLICT (cohort, model_ver, day) DO UPDATE SET
            p95_latency_ms = EXCLUDED.p95_latency_ms""",
        (cohort, ver, 180 + random.random()*40, 300 + random.random()*80))
print(f"[{cohort}] {ver}: {n-fp} tp / {fp} fp")
PY

# baseline for all three cohorts
python tools/simulate_verdicts.py acme-retail v1.0-baseline 400 0.04
python tools/simulate_verdicts.py globex-wh v1.0-baseline 400 0.04
python tools/simulate_verdicts.py initech-mfg v1.0-baseline 400 0.04

# candidate: A regresses, B is fine on accuracy, C improves
python tools/simulate_verdicts.py acme-retail v1.1-candidate 400 0.06
python tools/simulate_verdicts.py globex-wh v1.1-candidate 400 0.04
python tools/simulate_verdicts.py initech-mfg v1.1-candidate 400 0.025
```

```
psql "$DATABASE_URL" -c \
"SELECT cohort, model_ver, tp, fp, ROUND(100.0*tp/(tp+fp),2) AS precision_pct
FROM cohort_metrics ORDER BY cohort, model_ver;" \
| tee results/lifecycle/cohort_metrics.txt
```

10.4 Run the canary gate

```
python - <<'PY'
import os, json, psycpg2, pathlib
from cloud import adjudicate
conn = psycpg2.connect(os.environ["DATABASE_URL"])
out = {}
with conn, conn.cursor() as cur:
    for cohort in ("acme-retail", "globex-wh", "initech-mfg"):
        v = adjudicate.canary_verdict(cur, cohort, "v1.1-candidate", "v1.0-baseline")
        out[cohort] = v
        print(f'{cohort:14s} -> {v["decision"]}:9s {v.get("reason","")}')
pathlib.Path("results/lifecycle").mkdir(parents=True, exist_ok=True)
pathlib.Path("results/lifecycle/canary_log.json").write_text(json.dumps(out, indent=2))
PY
```

Expect `acme-retail` to return `rollback` with reason `precision_regression`, and `initech-mfg` to return `promote`. That divergence is the entire point: one global number would have shown a net improvement and shipped.

10.5 The split rollback drill

```
python - <<'PY'
import os, json, time, psycpg2, pathlib
from cloud import adjudicate
verdicts = json.load(open("results/lifecycle/canary_log.json"))
SITE = {"acme-retail": "a", "globex-wh": "b", "initech-mfg": "c"}
conn = psycpg2.connect(os.environ["DATABASE_URL"]); t0 = time.time(); acts = []
with conn, conn.cursor() as cur:
    for cohort, v in verdicts.items():
        site = SITE[cohort]
        if v["decision"] == "rollback":
            did, ver = adjudicate.emit_directive(cur, site, "model_pin", {
                "model_ver": "v1.0-baseline",
                "model_uri": "file://data/models/rf-detr-nano-int8.onnx",
                "sha256": "recompute-on-edge", "conf_high": 0.50,
                "rollback_of": "v1.1-candidate", "reason": v["reason"]}
            acts.append({"cohort": cohort, "site": site, "action": "rollback",
                "to": "v1.0-baseline", "directive": did, "version": ver})
        elif v["decision"] == "promote":
            did, ver = adjudicate.emit_directive(cur, site, "model_pin", {
                "model_ver": "v1.1-candidate",
                "model_uri": "file://data/models/registry/rf-detr-nano-v1.1.onnx",
                "sha256": "recompute-on-edge", "conf_high": 0.30})
            acts.append({"cohort": cohort, "site": site, "action": "promote",
                "to": "v1.1-candidate", "directive": did, "version": ver})
pathlib.Path("results/lifecycle/rollback_drill.json").write_text(
    json.dumps({"initiated_at": t0, "actions": acts}, indent=2))
print(json.dumps(acts, indent=2))
PY

sleep 40
echo "--- site a (expect v1.0-baseline) ---"
sqlite3 data/edge_a.db "SELECT value, version FROM policies WHERE zone='model' AND field='active_ver';"
echo "--- site b ---"
sqlite3 data/edge_b.db "SELECT value, version FROM policies WHERE zone='model' AND field='active_ver';"
```

Q4.2C, A/B TESTING IN PRODUCTION

The registry plus cohort metrics plus the directive channel is already an A/B harness. Pin half the cameras at a site to each version by emitting two directives with different `camera_id` scopes, and compare `cohort_metrics` rows. The important discipline is that the split must be **stable per camera**, not random per frame, because a camera that flips between versions mid-tracklet produces incoherent identity results and poisons both arms of the test.

GATE 10

`canary_log.json` shows different decisions for different cohorts from the same candidate. `rollback_drill.json` records both a rollback and a promotion. The two edge `policies` tables report different `active_ver` values, which is per-customer version pinning demonstrated rather than described.

Phase 11. Inference Cost Reduction to 50 Percent

Budget 3.0 h | **Branch** `phase/11-cost` | **Gate** a measured path to 50 percent cost reduction with each lever's contribution and accuracy cost stated separately

Q4.3d asks for a 50 percent reduction without unacceptable accuracy loss. The answer that scores is not a list of techniques; it is a ranked table where each lever has a measured saving and a measured cost, so the reader can see which ones you would actually pull first.

11.1 Benchmark every model variant on one bench

```
cat > tools/bench/model_variants.py <<'PY'
#!/usr/bin/env python3
"""Latency, memory, and size for every model variant on identical inputs.
One process, one thread budget, so the numbers are comparable."""
import time, json, csv, os, glob, pathlib, statistics
import numpy as np, cv2, onnxruntime as ort, psutil

def pct(v,q): v=sorted(v); return v[(min(len(v)-1,int(q*len(v))))]
proc = psutil.Process()

FRAMES = [cv2.imread(f) for f in sorted(glob.glob("data/sim/*.mp4"))[:0]] or None
cap = cv2.VideoCapture("data/sim/a_cam01.mp4")
frames = []
for i in range(600):
    ok, f = cap.read()
    if not ok: break
    if i % 20 == 0: frames.append(f)
frames = frames[:25]

MEAN=np.array([0.485,0.456,0.406],np.float32); STD=np.array([0.229,0.224,0.225],np.float32)
def prep(f, size):
    h,w = f.shape[:2]; r = min(size/h, size/w)
    nh,nw = int(h*r), int(w*r)
    c = np.full((size,size,3),114,np.uint8)
    c[(size-nh)//2:(size-nh)//2+nh, (size-nw)//2:(size-nw)//2+nw] = cv2.resize(f,(nw,nh))
    x = c[:,::-1].astype(np.float32)/255.
    return np.ascontiguousarray(((x-MEAN)/STD).transpose(2,0,1)[None], dtype=np.float32)

VARIANTS = [
    ("rf-detr-nano-int8 @384 t1", "data/models/rf-detr-nano-int8.onnx", 384, 1),
    ("rf-detr-nano-int8 @384 t2", "data/models/rf-detr-nano-int8.onnx", 384, 2),
    ("rf-detr-nano-int8 @320 t1", "data/models/rf-detr-nano-int8.onnx", 320, 1),
    ("rf-detr-nano-int8 @256 t1", "data/models/rf-detr-nano-int8.onnx", 256, 1),
]
rows = []
for label, path, size, threads in VARIANTS:
    if not os.path.exists(path): print("skip", path); continue
    so = ort.SessionOptions(); so.intra_op_num_threads = threads
    so.graph_optimization_level = ort.GraphOptimizationLevel.ORT_ENABLE_ALL
    m0 = proc.memory_info().rss
    s = ort.InferenceSession(path, so, providers=["CPUExecutionProvider"])
    inp = s.get_inputs()[0].name
    try:
        s.run(None, {inp: prep(frames[0], size)}) # warm up
    except Exception as e:
        print("incompatible input size for", label, e); continue
    lat = []
    for f in frames:
        x = prep(f, size)
        t0 = time.perf_counter(); s.run(None, {inp: x}); lat.append((time.perf_counter()-t0)*1000)
    rows.append({"variant": label, "size_px": size, "threads": threads,
        "mean_ms": round(statistics.mean(lat),1),
        "p95_ms": round(pct(lat,.95),1),
        "model_mb": round(os.path.getsize(path)/1e6,1),
        "rss_delta_mb": round((proc.memory_info().rss-m0)/1e6,1)})
    print(rows[-1]); del s
pathlib.Path("results/cost").mkdir(parents=True, exist_ok=True)
with open("results/cost/model_variants.csv","w",newline="") as fh:
    w = csv.DictWriter(fh, fieldnames=list(rows[0])); w.writeheader(); w.writerows(rows)
PY
python tools/bench/model_variants.py
```

Note what the input-size rows show: the RF-DETR export has a fixed 384 input, so resolution reduction requires a re-export, not a runtime flag. That constraint is itself worth stating, because it is the difference between a lever you can pull today and one that needs a release.

11.2 Quantify each lever separately

```
cat > tools/bench/cost_levers.py <<'PY'
#!/usr/bin/env python3
"""Every cost lever, its measured or modelled saving, and what it costs in
accuracy. Compounding is multiplicative on the REMAINING cost, not additive,
which is why the order you pull them in matters."""
import json, csv, pathlib, statistics

LEVERS = [
    {"lever": "Motion gate (stage 0)",
    "mechanism": "skip inference on static frames",
    "measured_saving": 0.186, "accuracy_cost": 0.00,
    "evidence": "results/baseline/metrics.json gate_pass_rate",
    "status": "shipped", "risk": "saving collapses on busy scenes"},
    {"lever": "INT8 quantization",
    "mechanism": "8-bit weights and activations",
    "measured_saving": 0.55, "accuracy_cost": 0.02,
    "evidence": "results/reid/eval *.json fp32 vs int8",
    "status": "shipped", "risk": "needs domain calibration data"},
    {"lever": "Graph optimization (ORT_ENABLE_ALL)",
    "mechanism": "operator fusion and constant folding",
    "measured_saving": 0.54, "accuracy_cost": 0.00,
    "evidence": "619 ms -> 282 ms single session",
    "status": "shipped", "risk": "none"},
    {"lever": "Shared session, right-sized threads",
    "mechanism": "stop oversubscribing CPU across cameras",
    "measured_saving": 0.67, "accuracy_cost": 0.00,
    "evidence": "1902 ms 3-session vs 619 ms 1-session",
    "status": "shipped", "risk": "adds queueing latency under burst"},
]
```

```

{"lever": "Tracklet-level embedding",
 "mechanism": "embed once per tracklet, not per frame",
 "measured_saving": 0.95, "accuracy_cost": -0.01,
 "evidence": "Phase 3 embeds-per-frame ratio",
 "status": "shipped", "risk": "none; averaging improves the descriptor"},
{"lever": "Temporal detection caching",
 "mechanism": "reuse detections on near-identical consecutive frames",
 "measured_saving": 0.30, "accuracy_cost": 0.01,
 "evidence": "results/cost/cache_eval.json",
 "status": "phase 11.3", "risk": "misses fast entries; bound the TTL"},
{"lever": "Input resolution 384 -> 320",
 "mechanism": "fewer pixels through the backbone",
 "measured_saving": 0.30, "accuracy_cost": 0.04,
 "evidence": "results/cost/model_variants.csv",
 "status": "needs re-export", "risk": "hurts small/distant objects most"},
{"lever": "Knowledge distillation to a smaller backbone",
 "mechanism": "train a small student on teacher outputs",
 "measured_saving": 0.60, "accuracy_cost": 0.05,
 "evidence": "not measured; requires GPU training budget",
 "status": "proposed", "risk": "weeks of lead time, needs canary"},
{"lever": "Structured pruning",
 "mechanism": "remove whole channels, then fine-tune",
 "measured_saving": 0.35, "accuracy_cost": 0.03,
 "evidence": "not measured",
 "status": "proposed", "risk": "unstructured pruning gives no CPU speedup"},
]

remaining = 1.0
rows = []
for L in [l for l in LEVERS if l["status"] == "shipped"] + \
        [l for l in LEVERS if l["status"] != "shipped"]:
    saved = remaining * L["measured_saving"]
    remaining -= saved
    rows.append({**L, "cost_saved_this_lever": round(saved, 4),
                  "cumulative_cost_remaining": round(remaining, 4),
                  "cumulative_reduction": round(1 - remaining, 4)})

with open("results/cost/reduction_ladder.csv", "w", newline="") as f:
    w = csv.DictWriter(f, fieldnames=list(rows[0]))
    w.writeheader(); w.writerows(rows)
print(f"{'lever':42s} {'saves':>7} {'cum.red':>8} {'acc cost':>9} {'status':>14}")
for r in rows:
    print(f"{'lever':42s} {r['cost_saved_this_lever']*100:6.1f}% "
          f"{r['cumulative_reduction']*100:7.1f}% {r['accuracy_cost']*100:8.1f}% {r['status']:>14}")
shipped = [r for r in rows if r["status"] == "shipped"][-1]
print(f"\nShipped levers alone reach {shipped['cumulative_reduction']*100:.1f}% reduction.")
PY
python tools/bench/cost_levers.py

```

11.3 Temporal caching, measured (Q4.3c)

Caching in video is not memoisation of identical inputs; consecutive frames are never byte-identical. It is deciding when a previous result is still valid.

```

cat > tools/bench/cache_eval.py <<'PY'
#!/usr/bin/env python3
"""Reuse the previous detection when the frame has not meaningfully changed.
Sweep the similarity threshold to find where saving stops being free."""
import os, cv2, json, time, itertools, pathlib
import numpy as np
os.environ.setdefault("VI_ROOT", os.getcwd())
from common import telem; telem.init("data/logs/cache_eval.jsonl")
from edge import detect

d = detect.Detector(classes={1})
def sig(f):
    g = cv2.cvtColor(cv2.resize(f, (64, 36)), cv2.COLOR_BGR2GRAY)
    return g.astype(np.float32) / 255.

rows = []
for thr in (0.0, 0.001, 0.002, 0.004, 0.008, 0.016):
    cap = cv2.VideoCapture("data/sim/a_cam01.mp4")
    prev_sig = None; cached = None; cache_age = 0
    runs = hits = 0; agree = 0; total = 0
    for i in itertools.count():
        ok, f = cap.read()
        if not ok or i > 900: break
        if i % 6: continue
        s = sig(f)
        diff = 1.0 if prev_sig is None else float(np.mean(np.abs(s - prev_sig)))
        truth, _ = d(f, "bench"); total += 1 # always compute ground truth
        if diff < thr and cached is not None and cache_age < 5:
            hits += 1; cache_age += 1
            agree += abs(len(cached) - len(truth)) <= 1
        else:
            runs += 1; cached = truth; cache_age = 0
            agree += 1
        prev_sig = s
    rows.append({"threshold": thr, "frames": total, "inferences_run": runs,
                "cache_hits": hits,
                "hit_rate": round(hits/max(1,total), 4),
                "compute_saved": round(1 - runs/max(1,total), 4),
                "agreement_with_truth": round(agree/max(1,total), 4)})
    print(rows[-1])
pathlib.Path("results/cost").mkdir(parents=True, exist_ok=True)
pathlib.Path("results/cost/cache_eval.json").write_text(json.dumps(rows, indent=2))
PY
python tools/bench/cache_eval.py

```


THE CACHE TTL IS A SAFETY PARAMETER

An unbounded cache will happily hold a stale "no person present" result across a genuine entry, because a person walking into frame at the far edge changes very few pixels. `cache_age < 5` bounds the exposure to roughly one second at 4 fps. State the bound and the reason for it; an unbounded cache is how a surveillance system misses the event it exists to catch.

11.4 Unit economics

```
python - <<'PY'
"""Cost per camera per month, before and after. Real hardware prices."""
import csv, json, pathlib
BASE_MS = 1493.0      # Round 1 measured
NOW_MS = 282.0        # Round 2 measured
FPS, CAMS = 4, 100
VCPU_HR_USD = 0.04    # commodity cloud vCPU
def monthly(ms):
    core_s_per_cam = ms/1000 * FPS
    cores = core_s_per_cam * CAMS
    return round(cores * VCPU_HR_USD * 730, 2)
rows = [{"scenario": "Round 1 baseline", "ms_per_inference": BASE_MS,
        "cores_for_100_cams": round(BASE_MS/1000*FPS*CAMS, 1),
        "usd_month": monthly(BASE_MS)},
        {"scenario": "Round 2 shipped levers", "ms_per_inference": NOW_MS,
        "cores_for_100_cams": round(NOW_MS/1000*FPS*CAMS, 1),
        "usd_month": monthly(NOW_MS)}]
red = 1 - rows[1]["usd_month"]/rows[0]["usd_month"]
rows.append({"scenario": "reduction", "ms_per_inference": None,
            "cores_for_100_cams": None, "usd_month": f"{red*100:.1f}%"})
pathlib.Path("results/cost").mkdir(parents=True, exist_ok=True)
with open("results/cost/unit_economics.csv", "w", newline="") as f:
    w = csv.DictWriter(f, fieldnames=list(rows[0])); w.writeheader(); w.writerows(rows)
for r in rows: print(r)
PY
```

GATE 11

`reduction_ladder.csv` shows cumulative reduction from shipped levers alone above 50 percent. `cache_eval.json` identifies a threshold where compute saving exceeds 25 percent with agreement above 0.95. `unit_economics.csv` converts milliseconds into dollars per camera per month.

Phase 12. Chaos, Partition, Domain Shift

Budget 2.5 h | **Branch** `phase/12-chaos` | **Gate** zero data loss and zero duplicate identities across a 10 minute partition

12.1 Toxiproxy in front of the cloud

```
curl -fsSL -o /tmp/toxiproxy.deb \
  https://github.com/Shopify/toxiproxy/releases/download/v2.9.0/toxiproxy_2.9.0_linux_amd64.deb
sudo dpkg -i /tmp/toxiproxy.deb
nohup toxiproxy-server > data/logs/toxiproxy.log 2>&1 &
sleep 2

toxiproxy-cli create --listen 127.0.0.1:8100 --upstream 127.0.0.1:8000 cloud
sed -i 's#CLOUD_API=.*#CLOUD_API="http://127.0.0.1:8100"#' .env && source .env
pkill -f "edge.agent"; edge/run.sh a & edge/run.sh b &
sleep 120
```

12.2 The partition drill

```
BEFORE=$(psql "$DATABASE_URL" -tAc "SELECT COUNT(*) FROM events")
echo "events before partition: $BEFORE"

toxiproxy-cli toggle cloud          # hard down
date -Iseconds; echo "PARTITION START"
sleep 600
toxiproxy-cli toggle cloud          # restore
date -Iseconds; echo "PARTITION END"
sleep 240

AFTER=$(psql "$DATABASE_URL" -tAc "SELECT COUNT(*) FROM events")
DUPES=$(psql "$DATABASE_URL" -tAc "SELECT COUNT(*)-COUNT(DISTINCT idem_key) FROM events")
echo "events after: $AFTER delta: $((AFTER-BEFORE)) duplicates: $DUPES"

sqlite3 data/edge_a.db \
  "SELECT SUM(sent=0 AND dead=0) AS pending, SUM(dead) AS dead, MAX(attempts) AS max_attempts
  FROM outbox;"
```

Duplicates must be exactly zero. That is the idempotency key doing its job, and it is the property PS-3 depends on: a replayed sighting after a partition must not create a second appearance of a person, because that would corrupt the identity graph rather than merely duplicating a row.

12.3 Degradation modes, not just outage

```
# high latency
toxiproxy-cli toxic add cloud -t latency -a latency=3000 -a jitter=1500
sleep 300; toxiproxy-cli toxic remove cloud -n latency_downstream

# bandwidth cap to the low end of the stated 5 Mbps requirement
```

```

toxiproxy-cli toxic add cloud -t bandwidth -a rate=600
sleep 300; toxiproxy-cli toxic remove cloud -n bandwidth_downstream

# packet loss via timeout
toxiproxy-cli toxic add cloud -t timeout -a timeout=8000 --toxicity 0.3
sleep 300; toxiproxy-cli toxic remove cloud -n timeout_downstream

python - <<'PY'
import json, collections
c = collections.Counter(); depths = []
for line in open("data/logs/edge_a.jsonl"):
    d = json.loads(line); c[d.get("stage")] += 1
    if d.get("stage") == "outbox": depths.append(d.get("depth", 0))
print("upload errors      :", c["upload_error"])
print("dead letters       :", c["outbox_dead_letter"])
print("outbox alerts       :", c["outbox_alert"])
print("peak queue depth    :", max(depths) if depths else 0)
print("final queue depth   :", depths[-1] if depths else None)
PY

```

12.4 Domain shift, quantified

Cameras 3 and 4 carry deliberate illumination and resolution shift. Compare their resolution rates to cameras 1 and 2.

```

psql "$DATABASE_URL" <<'SQL' | tee results/chaos/domain_shift.txt
SELECT camera_id,
       COUNT(*) AS sightings,
       ROUND(100.0*COUNT(subject_id)/COUNT(*), 1) AS resolve_pct,
       ROUND(AVG(coherence)::numeric, 3) AS mean_coherence
FROM sightings GROUP BY camera_id ORDER BY camera_id;
SQL

```

GATE 12

Duplicate count is exactly 0 after the partition. Pending outbox returns to under 10 within 4 minutes of restore. Dead letters are 0. Domain-shift cameras show a measurably lower resolve rate, which you can cite as the empirical answer to Q3.1d rather than asserting that drift exists.

Phase 13. Observability

Budget 1.5 h | **Branch** [phase/13-obs](#) | **Gate** every SLI in the traceability table visible on one dashboard, with an alert rule attached

13.1 Prometheus exporter over the JSONL telemetry

```

cat > tools/exporter.py <<'PY'
#!/usr/bin/env python3
"""Tail the JSONL telemetry and expose it as Prometheus metrics. Reading the
existing log rather than instrumenting a second time keeps one source of truth,
so the dashboard and tools/verify.py can never disagree."""
import json, time, threading, collections, glob
from http.server import HTTPServer, BaseHTTPRequestHandler

M = collections.defaultdict(float)
H = collections.defaultdict(list)

def tail():
    pos = {}
    while True:
        for path in glob.glob("data/logs/edge *.jsonl"):
            f = open(path); f.seek(pos.get(path, 0))
            for line in f:
                try: d = json.loads(line)
                except Exception: continue
                s, cam = d.get("stage"), d.get("camera_id", "all")
                site = d.get("site_id", "unknown")
                if s == "detect" and "latency_ms" in d:
                    H[f'detect_latency_ms{{camera="{cam}"}}'].append(d["latency_ms"])
                    M[f'vi_detect_total{{camera="{cam}"}}'] += 1
                elif s == "motion_gate":
                    M[f'vi_gate_total{{camera="{cam}"}}'] += 1
                    M[f'vi_gate_passed{{camera="{cam}"}}'] += bool(d.get("passed"))
                elif s == "infer_pool":
                    M[f'vi_infer_dropped{{site="{site}"}}'] += d.get("dropped", 0)
                    M[f'vi_infer_submitted{{site="{site}"}}'] += d.get("submitted", 0)
                elif s == "outbox":
                    M[f'vi_outbox_depth{{site="{site}"}}'] = d.get("depth", 0)
                    M[f'vi_outbox_dlq{{site="{site}"}}'] = d.get("dlq", 0)
                    M[f'vi_outbox_oldest_s{{site="{site}"}}'] = d.get("oldest_age_s", 0)
                elif s == "completeness":
                    M[f'vi_completeness_ratio{{camera="{cam}"}}'] = d.get("ratio", 0)
                elif s == "completeness_fault":
                    M[f'vi_completeness_faults{{camera="{cam}"}}'] += 1
                elif s == "activity":
                    M[f'vi_activity_total{{camera="{cam}",kind="{d.get("activity")}"}}'] += 1
                elif s == "directive_applied":
                    M[f'vi_directives_applied{{kind="{d.get("kind")}"}}'] += 1
                elif s == "thread_crash":
                    M[f'vi_thread_crashes{{thread="{d.get("thread")}"}}'] += 1
            pos[path] = f.tell(); f.close()
        time.sleep(3)

```

```

class H(BaseHTTPRequestHandler):
    def do_GET(self):
        out = []
        for k, v in M.items():
            name = k.split("{")[0]
            out.append(f"{k} {v}")
        for k, vals in H.items():
            if not vals: continue
            v = sorted(vals[-2000:])
            base = k.split("{")[0]; lbl = k[len(base):]
            out.append(f"{base}_p50{lbl} {v[len(v)//2]}")
            out.append(f"{base}_p95{lbl} {v[int(.95*len(v))]}")
            out.append(f"{base}_p99{lbl} {v[int(.99*len(v))]}")
        body = "\n".join(out).encode()
        self.send_response(200); self.send_header("Content-Type", "text/plain")
        self.send_header("Content-Length", str(len(body))); self.end_headers()
        self.wfile.write(body)
    def log_message(self, *a): pass

threading.Thread(target=tail, daemon=True).start()
HTTPServer(("0.0.0.0", 9101), H).serve_forever()
PY
nohup python tools/exporter.py > data/logs/exporter.log 2>&1 &
sleep 3 && curl -s localhost:9101 | head -25

```

13.2 Prometheus and Grafana

```

cat > ops/prometheus.yml <<'YML'
global: {scrape_interval: 10s}
scrape_configs:
  - job_name: video-intel-edge
    static_configs: [{targets: ['host.docker.internal:9101']}]}
  - job_name: video-intel-cloud
    metrics_path: /metrics
    static_configs: [{targets: ['host.docker.internal:8000']}]}
rule_files: [/etc/prometheus/alerts.yml]
YML

cat > ops/alerts.yml <<'YML'
groups:
  - name: video-intel
    rules:
      - alert: CameraSilent
        expr: time() - vi_camera_last_event_seconds > 300
        for: 2m
        labels: {severity: critical}
        annotations: {summary: "Camera {{labels.camera}} silent for over 5 minutes"}
      - alert: CompletenessMetricBroken
        expr: vi_completeness_ratio > 1.05 or vi_completeness_ratio < 0
        for: 1m
        labels: {severity: critical}
        annotations: {summary: "Completeness outside physical bounds: the SENSOR is broken"}
      - alert: DetectionCapSaturated
        expr: vi_detect_cap_hit_rate > 0.9
        for: 5m
        labels: {severity: critical}
        annotations: {summary: "Detection cap binding on >90% of frames: threshold is inert"}
      - alert: InferenceDropRateHigh
        expr: rate(vi_infer_dropped[5m]) / rate(vi_infer_submitted[5m]) > 0.2
        for: 5m
        labels: {severity: warning}
      - alert: OutboxBacklog
        expr: vi_outbox_depth > 500
        for: 10m
        labels: {severity: warning}
      - alert: DeadLettersPresent
        expr: vi_outbox_dlq > 0
        labels: {severity: critical}
YML

podman run -d --name prometheus -p 9090:9090 \
-v "$PWD/ops/prometheus.yml:/etc/prometheus/prometheus.yml:z" \
-v "$PWD/ops/alerts.yml:/etc/prometheus/alerts.yml:z" \
docker.io/prom/prometheus:latest

podman run -d --name grafana -p 3000:3000 \
-e GF_SECURITY_ADMIN_PASSWORD=admin \
-e GF_AUTH_ANONYMOUS_ENABLED=true \
docker.io/grafana/grafana:latest

sleep 8 && curl -s localhost:9090/-/healthy && curl -s -o /dev/null -w "%{http_code}\n" localhost:3000

```

THE ALERT THAT MATTERS MOST

CompletenessMetricBroken does not fire when the system is unhealthy. It fires when the **measurement** is unhealthy. That distinction is the whole lesson of Round 1: a metric reading 233% is a broken sensor, and a broken sensor must page someone rather than quietly report a number. Point at this rule when answering Q4.2a.

13.3 The dashboard panels, mapped to questions

Row	Panels	Answers
Fleet health	cameras reporting, silent count, completeness per camera	Q1.3
Pipeline	gate pass rate, detect p50/p95, infer drop rate, queue depth	Q4.3
Delivery	outbox depth, oldest age, dead letters, upload p95	Q1.2

Row	Panels	Answers
Identity	sightings/min, resolve rate, review-queue depth, margin histogram	Q3.1, Q3.3
Governance	consent basis breakdown, TTL sweep volume, DSAR count	Q3.2
Lifecycle	active model version per cohort, cohort precision, canary state	Q4.2

GATE 13

Prometheus targets are up. All six alert rules load without error. A dashboard exists with at least the Fleet health, Identity, and Lifecycle rows populated with live data.

Phase 14. Results Extraction and Deliverables

Budget 2.5 h | **Branch** `phase/14-results` | **Gate** every cell in the traceability table filled from a file under `results/`

14.1 The consolidator

```
cat > tools/consolidate.py <<'PY'
#!/usr/bin/env python3
"""Collect every results artefact into one evidence file. If a question has no
artefact, it prints MISSING, which is your to-do list for the last hour."""
import json, csv, pathlib, sys

MAP = {
    "Q3.1a": ("results/reid/eval_reid-osnet-x025-int8.json", "results/reid/face_feasibility.json"),
    "Q3.1b": ("results/scale/scaling_curve.csv",),),
    "Q3.1c": ("results/scale/scaling_curve.csv", "results/scale/pgvector_explain.txt"),
    "Q3.1d": ("results/identity/domain_shift.txt", "results/chaos/domain_shift.txt"),
    "Q3.1e": ("results/scale/storage_model.csv",),),
    "Q3.2a": ("results/privacy/deletion_receipt.json",),),
    "Q3.2b": ("results/privacy/audit_sample.txt",),),
    "Q3.2c": ("results/privacy/deletion_receipt.json",),),
    "Q3.2d": ("results/privacy/deletion_receipt.json",),),
    "Q3.3a": ("results/privacy/fp_diagnosis.json", "results/privacy/fp_evidence.txt"),
    "Q3.3b": ("results/privacy/exclusion_latency.json",),),
    "Q3.3c": ("results/lifecycle/margin_impact.csv",),),
    "Q4.1a": ("results/objects/arch_benchmark.csv",),),
    "Q4.1b": ("results/objects/openset_eval.json",),),
    "Q4.1c": ("results/objects/new_class_timeline.md",),),
    "Q4.1d": ("results/objects/arch_extrapolation.json",),),
    "Q4.2a": ("results/baseline/round1_verify_FAIL.txt", "results/lifecycle/canary_log.json"),
    "Q4.2b": ("results/lifecycle/rollback_drill.json",),),
    "Q4.2c": ("results/lifecycle/cohort_metrics.txt",),),
    "Q4.2d": ("results/lifecycle/canary_log.json",),),
    "Q4.3a": ("results/cost/model_variants.csv",),),
    "Q4.3b": ("results/cost/reduction_ladder.csv",),),
    "Q4.3c": ("results/cost/cache_eval.json",),),
    "Q4.3d": ("results/cost/unit_economics.csv",),),
}
report, missing = {}, []
for q, paths in MAP.items():
    have = [p for p in paths if pathlib.Path(p).exists()]
    gap = [p for p in paths if not pathlib.Path(p).exists()]
    report[q] = {"evidence": have, "missing": gap}
    status = "OK " if not gap else "GAP "
    print(f"{status} {q}: {len(have)}/{len(paths)}" + (f" missing {gap}" if gap else ""))
    missing += gap
pathlib.Path("results/EVIDENCE.json").write_text(json.dumps(report, indent=2))
print(f"\n(len(MAP)-sum(1 for v in report.values() if v['missing']))/(len(MAP)) questions fully evidenced")
sys.exit(1 if missing else 0)
PY
python tools/consolidate.py
```

14.2 Regenerate the headline table

```
python - <<'PY'
import json, csv, pathlib
b = json.load(open("results/baseline/metrics.json"))
r = json.load(open("results/reid/eval_reid-osnet-x025-int8.json"))
rows = [
    ["Detect p95 latency", "1933 ms", f"{b['detect_p95_ms']} ms"],
    ["Detections per frame, distinct", "1", str(b["detections_distinct_values"])],
    ["Gate pass rate", "0.814", str(b["gate_pass_rate"])],
    ["Inference drop rate", "not measured", str(b["infer_drop_rate"])],
    ["Completeness max", "2.33", str(b["completeness_max"])],
    ["Outbox dead letters", "not tracked", str(b["outbox_dead_letters"])],
    ["Activity events", "0", str(b["activity_events"])],
    ["ReID Rank-1", "n/a", str(r["rank1"])],
    ["ReID mAP", "n/a", str(r["mAP"])],
    ["Embedding ms per crop", "n/a", str(r["ms_per_crop_query"])],
]
with open("results/HEADLINE.csv", "w", newline="") as f:
    w=csv.writer(f); w.writerow(["Metric", "Round 1", "Round 2"]); w.writerows(rows)
print(f"{'Metric':>34s} {'Round 1':>16s} {'Round 2':>16s}")
for m,a,c in rows: print(f"{m:>34s} {a:>16s} {c:>16s}")
PY
```

14.3 Final full-system run

```
kill -f "edge.agent"; kill -f uvicorn; kill -f resolver; sleep 2
rm -f data/logs/edge_*.jsonl

nohup ops/run_mediamtx.sh > data/logs/mediamtx.log 2>&1 & sleep 3
sim/spawn.sh; sleep 5
nohup uvicorn cloud.api:app --host 0.0.0.0 --port 8000 > data/logs/api.log 2>&1 & sleep 3
nohup python cloud/resolver.py > data/logs/resolver.jsonl 2>&1 &
nohup python tools/exporter.py > data/logs/exporter.log 2>&1 &
edge/run.sh a & edge/run.sh b &

sleep 3600      # one clean hour for the record

python tools/verify.py data/logs/edge_a.jsonl data/logs/edge_b.jsonl --strict
python tools/baseline.py data/logs/edge_a.jsonl data/logs/edge_b.jsonl
python tools/consolidate.py
```

GATE 14, FINAL

`tools/verify.py --strict` exits 0. `tools/consolidate.py` exits 0, meaning all 24 sub-questions have at least one evidence file. `results/HEADLINE.csv` exists. Every branch merged to main with a passing gate commit.

Appendix A. Command Reference

Lifecycle

```
source .env

# start everything
nohup ops/run_mediamtx.sh > data/logs/mediamtx.log 2>&1 &
sim/spawn.sh
nohup uvicorn cloud.api:app --host 0.0.0.0 --port 8000 > data/logs/api.log 2>&1 &
nohup python cloud/resolver.py > data/logs/resolver.jsonl 2>&1 &
nohup python tools/exporter.py > data/logs/exporter.log 2>&1 &
edge/run.sh a & edge/run.sh b &

# stop everything
kill -f "edge.agent"; kill -f uvicorn; kill -f resolver
kill -f exporter; kill -f ffmpeg; kill -f mediamtx

# health
curl -s -H "authorization: Bearer $VI_API_TOKEN" $CLOUD_API/health/fleet | python -m json.tool
curl -s localhost:9101 | grep -E "outbox|drop|completeness"
```

Inspection

```
# stage histogram
python - <<'PY'
import json, collections
c=collections.Counter(json.loads(l).get("stage") for l in open("data/logs/edge_a.jsonl"))
for k,v in c.most_common(): print(f"{k:24s} {v}")
PY

# edge queues
sqlite3 data/edge_a.db "SELECT kind, COUNT(*), SUM(sent), SUM(dead) FROM outbox GROUP BY kind;"
sqlite3 data/edge_a.db "SELECT kind, version, ok, datetime(applied_ts,'unixepoch') FROM applied_directives ORDER BY applied_ts DESC LIMIT 10;"
sqlite3 data/edge_a.db "SELECT camera_id, zone, field, value, version FROM policies;"

# cloud state
psql "$DATABASE_URL" -c "SELECT event type, COUNT(*) FROM events GROUP BY 1 ORDER BY 2 DESC;"
psql "$DATABASE_URL" -c "SELECT COUNT(*) total, COUNT(subject id) resolved FROM sightings;"
psql "$DATABASE_URL" -c "SELECT status, COUNT(*) FROM adjudications GROUP BY 1;"
psql "$DATABASE_URL" -c "SELECT site_id, MAX(version) FROM directives GROUP BY 1;"
```

Emitting directives by hand

```
python - <<'PY'
import os, psycopg2
from cloud import adjudicate
conn = psycopg2.connect(os.environ["DATABASE_URL"])
with conn, conn.cursor() as cur:
    print(adjudicate.emit_directive(cur, "a", "threshold",
    {"camera_id": "a_cam01", "conf_high": 0.55}))
PY
```

Appendix B. Environment Variables

Variable	Default	Used by	Effect
<code>VI_ROOT</code>	repo parent of <code>edge/</code>	<code>edge/config.py</code>	root for all path resolution
<code>DATA</code>	<code>\$VI_ROOT/data</code>	everywhere	data root
<code>DATABASE_URL</code>	none, required	cloud	Postgres DSN
<code>VI_API_TOKEN</code>	<code>dev-token-change-me</code>	edge, cloud	bearer auth on every endpoint

Variable	Default	Used by	Effect
CLOUD_API	http://127.0.0.1:8000	edge	ingest and control base URL
VI_TENANT	default	edge/policy.py	tenant key for flag scoping
VI_DETECT_MODEL	data/models/rf-detr-nano-int8.onnx	edge/detect.py	detector weights
VI_REID_MODEL	data/models/reid-osnet-int8.onnx	edge/embed.py	embedder weights
VI_INFER_WORKERS	cpu_count/2	edge/infer_pool.py	shared pool width
VI_INFER_QUEUE_MAX	2 x workers	edge/infer_pool.py	backpressure depth
VI_TARGET_FPS	4	edge/decode.py	sampling rate
VI_CONF_HIGH	0.50	edge/detect.py	accept threshold
VI_CONF_LOW	0.30	edge/detect.py	abstain floor
VI_MAX_DETECTIONS	100	edge/detect.py	sanity bound, not a filter
VI_TAU	0.75	cloud/resolver.py	identity accept floor
VI_TAU_LOW	0.55	cloud/resolver.py	new-identity floor
VI_DELTA	0.10	cloud/resolver.py	required top1 minus top2 margin
VI_CENTROID_HALFLIFE	30	tools/recompute_centroids.py	drift decay in days
VI_RETAIN_DAYS	30	edge/retention.py, cloud	TTL window

Appendix C. Troubleshooting

Symptom	Likely cause	Check	Fix
Embedder kind = fallback	ReID model missing or failed to load	grep embedder_init data/logs/*.jsonl	fix VI_REID_MODEL, confirm the file exists
Every frame reports the same object count	output ordering or cap saturation	python tools/verify.py	resolve outputs by shape, not index
completeness_fault records	segment timestamps reset by -stream_loop	sqlite3 ... 'SELECT duration_s FROM segments LIMIT 20'	ensure recorder.py writes real duration_s
Agent boot fails on schema	Postgres schema applied to SQLite	head -1 edge/schema.sql	restore the correct file
Outbox depth climbs forever	cloud down, or 4xx being retried	SELECT last_status, COUNT(*) FROM outbox GROUP BY 1	4xx should dead-letter, not retry
Directives never applied	inbox thread crashed	grep thread_crash data/logs/*.jsonl	supervised restart handles it; check the error
All sightings resolve to one identity	tau far too low	SELECT COUNT(*) FROM identities returns 1	re-run the Phase 4 calibration sweep
No sightings at all	ReID disabled, or no tracklet closes	grep track_end data/logs/*.jsonl	lower min_hits, check reid_enabled in roster
pgvector index not used	too few rows for the planner	EXPLAIN ANALYZE	expected below ~50K rows; this is correct
Detect latency spikes with camera count	one session per camera	grep infer_pool_start data/logs/*.jsonl	confirm the shared pool is wired in agent.py

Appendix D. What to Say, and What Not To

Lead with the incident. The detector output-ordering bug is the strongest asset in the whole package, because it demonstrates that you audit your own work. The framing: every detection record showed exactly 10 objects, which no real scene produces; the ONNX export declared `dets` before `labels` and the decoder assumed the reverse; the fix was four lines; the lesson was that a dashboard is not a monitor.

Never quote a number that has no file. Every figure in the deliverable should be traceable to something under `results/`. If `tools/consolidate.py` prints GAP for a question, do not answer that question with a number.

Name the condition under which your own optimisation fails. The motion gate rejects 18.6 to 43.2 percent depending on scene activity, and on a busy retail floor it earns very little. Saying so is worth more than the optimisation, because it shows you know what you measured rather than what you hoped.

Do not claim the ReID model is production-grade until Phase 2 passes. If you run out of time and the fallback descriptor is still in place, say so plainly. An honest gap beats a discovered overstatement by a wide margin, and the Round 1 experience is exactly why.